



INSTITUTO POLITÉCNICO NACIONAL

CENTRO DE ESTUDIOS CIENTÍFICOS Y TECNOLÓGICOS
N.9

“JUAN DE DIOS BÁTIZ”

TESINA

QUE PARA OBTENER EL TÍTULO DE:

TÉCNICO EN PROGRAMACIÓN

PRESENTAN:

Hernández Carmona Luis Emiliano	2024090379
Ruiz Vázquez Máximo	2024090711
Salinas Ramos Luis Miguel	2024090763
Correa Estrada Arantxa	2024090127
Velázquez López Romario	2024090857

ASESORES:



CDMX 27 DE FEBRERO DE 2026

Índice

Abstract.....	5
Resumen	5
Introducción.....	5
Capítulo 1: Particularidades del Software	6
Problemática	6
Justificación.....	7
Objetivos del Software.....	8
-Objetivo general	8
-Objetivos específicos	8
Alcance	8
Limitaciones	9
Planeación estratégica	9
-Misión.	9
-Visión.	10
-Logo.....	10
-Eslogan.....	10
-Filosofía.....	11
-Valores.	11
-FODA.....	12
Capítulo 2 Fundamento Teórico	13
Antecedentes.....	13
Definición de términos básicos	15
Definición Operacional	16
Unidad Docente.....	16
Unidad Experiencia de Usuario.....	19
Unidad Arquitectura del software.....	20
Capítulo 3 “Metodología de Desarrollo”	32
Viabilidad	32
Factibilidad	33
Rentabilidad	34
Backlog	35
Artefactos	38

Roles	38
Diagrama de Gantt:.....	39
Ruta Crítica:	39
Metodología SCRUM.....	40
Planeaciones de Sprint	41
Reportes de Sprint	42
Sprint 1.....	42
Sprint 2.....	55
Sprint 3.....	62
Ceremonias.....	69
Ceremonia – Sprint 1	69
Ceremonia – Sprint 2.....	70
Ceremonia – Sprint 3.....	71
Capítulo 4: Resultados	72
Prueba de Usuario	72
Tipo de Prueba: Alfa	72
Características de la Prueba	72
Ciclo de Deming Aplicado a MaxGrade.....	73
Planificar	73
Hacer.....	74
Verificar	75
Actuar	75
Conclusiones	76
Referencias.	78
Anexos	80

Índice de Tablas

Tabla 1 FODA.....	12
Tabla 2 Tipos de docente	16
Tabla 3 Estilo de enseñanza del docente	17
Tabla 4 Necesidades del docente	18
Tabla 5 Elementos de UX	19
Tabla 6 impactos positivos vs negativos	32
Tabla 7 Factibilidad Económica Costos Del Servidor.....	33
Tabla 8 Factibilidad Económica Costo Personal.....	33

Tabla 9 Factibilidad Financiera.....	34
Tabla 10 Fechas Sprint	41
Tabla 11 Ambientes De Prueba.....	41
Tabla 12 Historias de Usuario Sprint 1	42
Tabla 13 Documentación Código Sprint 1.....	47
Tabla 14 Pruebas Sprint 1	50
Tabla 15 Errores Corregidos Sprint 1	53
Tabla 16 Historias de Usuario Sprint 2	55
Tabla 17 Documentación Código Sprint 2.....	58
Tabla 18 Pruebas Sprint 2	59
Tabla 19 Errores Corregidos Sprint 2	61
Tabla 20 Historias de Usuario Sprint 3	62
Tabla 21 Documentación Código Sprint 3.....	64
Tabla 22 Pruebas Sprint 3	65
Tabla 23 Errores Corregidos Sprint 3	67

Índice de imágenes

Ilustración 1 Logo Uxmal.....	10
Ilustración 2 Diagrama de Arquitectura del sistema	21
Ilustración 3 Organigrama.....	38
Ilustración 4 Diagrama de Gantt 1	39
Ilustración 5 Diagrama de Gantt 2	39
Ilustración 6 Actividades Ruta Crítica.....	40
Ilustración 7 Ruta crítica	40
Ilustración 8 Diseño sprint 1	46
Ilustración 9 Diseño Sprint 2	57
Ilustración 10 Diseño Sprint 3	64

Abstract

MaxGrade is a minimalist web and mobile application designed to reduce digital barriers faced by teachers aged 40–60 at CECyT 9. It simplifies task management, grading, and academic tracking through an intuitive, role-based interface developed using agile SCRUM methodology.

Resumen

MaxGrade es una aplicación web y móvil minimalista diseñada para reducir las barreras digitales de docentes de 40 a 60 años en el CECyT 9. Simplifica la gestión de tareas, calificaciones y seguimiento académico mediante una interfaz intuitiva desarrollada con metodología ágil SCRUM.

Introducción

La incorporación de herramientas digitales en la educación media superior mexicana ha avanzado de manera acelerada en los últimos años, impulsada en gran medida por la necesidad de adaptarse a entornos de enseñanza híbridos y remotos. Sin embargo, este proceso no siempre ha sido acompañado de la formación adecuada ni del diseño de herramientas accesibles para todos los perfiles docentes. Plataformas como Google Classroom o Microsoft Teams, aunque ampliamente adoptadas, presentan interfaces complejas que generan confusión, errores frecuentes y exclusión digital, especialmente entre los profesores de mayor edad.

El presente trabajo surge a partir de una problemática identificada en el plantel CECyT 9 del Instituto Politécnico Nacional, donde una encuesta aplicada a 30 docentes reveló que la mayoría enfrenta dificultades operativas al usar plataformas digitales: 21 de ellos reportaron problemas para configurar tareas, 20 cometieron errores por no comprender bien las herramientas disponibles, y solo 10 afirmaron sentirse seguros al utilizarlas. Estos hallazgos evidencian que el problema no radica únicamente en la falta de capacitación, sino también en el diseño poco intuitivo de las soluciones tecnológicas existentes.

Ante esta situación, el equipo de desarrollo Uxmal propone MaxGrade: una aplicación web y móvil minimalista orientada a docentes de entre 40 y 60 años, cuyo objetivo principal es mejorar la eficiencia en la evaluación formativa y sumativa del alumnado. La propuesta se fundamenta en principios de diseño centrado en el usuario, priorizando iconografía intuitiva, lenguaje claro en español, navegación simplificada y procesos de guardado confiables, características que las plataformas actuales no satisfacen de manera adecuada para este perfil de usuario.

El desarrollo de MaxGrade se llevó a cabo mediante la metodología ágil SCRUM, estructurado en tres sprints de trabajo iterativo que abarcaron desde la implementación de los módulos de autenticación, gestión de clases y calificación, hasta funcionalidades avanzadas como la detección de uso de inteligencia artificial, la retroalimentación por voz y la publicación programada

de tareas. Todo el proceso fue documentado y validado a través de pruebas funcionales por módulo, con corrección de defectos dentro de cada ciclo de entrega.

El presente documento está organizado en capítulos que cubren, de manera progresiva, las particularidades del software desarrollado, los fundamentos teóricos que sustentan las decisiones de diseño y tecnología, y la descripción metodológica del proceso de construcción de la aplicación. Se incluyen también los antecedentes de proyectos similares, el marco conceptual empleado, así como los resultados obtenidos durante las fases de prueba. El objetivo final es demostrar que es posible construir tecnología educativa funcional, segura y verdaderamente accesible para los docentes que más lo necesitan.

Capítulo 1: Particularidades del Software

Problemática

El uso de plataformas digitales en las instituciones educativas mexicanas ha crecido de forma acelerada en los últimos años, sin embargo, esta adopción tecnológica no ha sido acompañada de la formación necesaria para los docentes. Tal como indican González et al. (2023), "los docentes juegan un papel fundamental en la integración efectiva de la tecnología en el aula, pero muchos no han recibido la capacitación necesaria para aprovechar al máximo su potencial" (p. 10). Esto provoca que tareas cotidianas como subir contenido, crear actividades o calificar a los alumnos representen una dificultad real, afectando tanto el trabajo del docente como el seguimiento académico de los estudiantes.

Ante esta situación surgen las siguientes preguntas: ¿por qué los docentes siguen teniendo dificultades operativas si las plataformas llevan años en uso? ¿Es un problema de capacitación o de diseño de las herramientas disponibles?

Es por ello, que en el plantel CECyT 9 se realizó una encuesta con una muestra de 30 docentes en un rango de edad de 30 a 60 años de una población de aproximadamente 60 docentes que ayudan a responder estas preguntas (Ver Anexo A), conforme a la encuesta realizada, 21 de los 30 docentes encuestados reporta dificultades al configurar tareas, 20 de los 30 cometen errores por no comprender bien las plataformas tales como Classroom o Teams (Siendo éstas las más comunes), 19 de los 30 se confunden con el orden de los pasos. En cuanto a la evaluación, 18 de los 30 señalan problemas al calificar porque no sabe si los cambios quedaron guardados. Solo 10 de 30 afirman sentirse seguros usando las plataformas anteriormente mencionadas y 18 de 30 las perciben como complicadas, lo que indica que el problema va más allá de la falta de capacitación.

Las fallas operativas reportadas por los docentes encuestados, donde la mayoría confiesa sentirse inseguro y confundido al usar herramientas digitales, coinciden con lo investigado por Pinzón y Mortis (2025). En su estudio, se destaca que factores como el "esfuerzo percibido" y las

"condiciones facilitadoras" son claves para que una persona use la tecnología con éxito; es decir, si la herramienta parece muy difícil o no existe un soporte técnico adecuado, se crean barreras que llevan a la exclusión digital. Esta situación no solo genera errores técnicos al calificar o subir tareas, sino que provoca un entorpecimiento de su labor profesional por un diseño que no se adapta a necesidades específicas como "ver iconos que parezcan objetos de la vida real y letras grandes que sean fáciles de leer" (Yañez Castillo, 2017), "tener pantallas ordenadas que no los confundan al buscar los trabajos de sus alumnos" (Rodríguez López, 2017), "contar con formas sencillas de guardar y organizar sus archivos sin que el proceso sea enredado" (Universidad Autónoma del Estado de Hidalgo [UAEH], 2022) y "recibir instrucciones con palabras simples, evitando términos técnicos o en inglés, para que puedan hacer las cosas por sí mismos sin depender de nadie" (Universidad Nacional Autónoma de México [UNAM], 2019).

Justificación

El uso de herramientas digitales en la labor docente es cada vez más creciente; sin embargo, muchos de los sistemas actuales presentan dificultades de uso que entorpecen el trabajo diario de los docentes. Esta situación provoca errores frecuentes al configurar actividades o registrar evaluaciones, lo que lleva a muchos profesores a preferir los métodos tradicionales para evitar confusiones.

La razón por la que se realiza este trabajo es para encontrar por qué las plataformas actuales no se adaptan a las necesidades de los maestros, lo que termina provocando una exclusión digital que afecta su bienestar. La utilidad para la sociedad es clara: al resolver estas dificultades tecnológicas, se ayuda a que los docentes trabajen de forma más eficiente y se garantiza que los alumnos reciban un mejor seguimiento en sus estudios. Con este proyecto se busca que la tecnología deje de ser un motivo de estrés y se convierta en una herramienta sencilla y segura que realmente apoye el proceso de enseñanza y aprendizaje.

Es importante buscar entender por qué los docentes, especialmente quienes tienen más años de experiencia, siguen enfrentando obstáculos para usar las plataformas digitales a pesar de que llevan años en las escuelas. Lo que esta investigación aporta de relevante es que permite ver que el problema no se queda solo en la falta de cursos de capacitación, sino que tiene mucho que ver con el diseño de las herramientas y el gran "esfuerzo percibido" que requieren para ser usadas. Al estudiar este tema, se atienden fallas reales como los errores al calificar, la confusión al seguir pasos técnicos y la inseguridad que sienten los profesores, situaciones que hoy en día entorpecen su labor profesional en planteles como el CECyT 9.

Este proyecto pretende ofrecer una alternativa que sea fácil de utilizar para los maestros.

Objetivos del Software

-Objetivo general

Desarrollar una aplicación tanto web como móvil, minimalista e intuitiva de trabajo escolar para el mejoramiento de la eficiencia de los docentes en un rango de 40 a 60 años en la evaluación formativa y sumativa de los alumnos.

-Objetivos específicos

- 1.- Diseñar una interfaz donde las herramientas y funciones estén organizadas de forma intuitiva para que el usuario las encuentre de forma rápida y lógica.
- 2.- Habilitar opciones de edición directa en los componentes, agilizando el proceso de corrección de errores en campos o especificaciones de la tarea creada.
- 3.-Programar un componente para la creación de rúbricas editables y estandarizadas con el fin de realizar la revisión de tareas para la calificación y retroalimentación de las tareas de los alumnos.

Alcance

El sistema está integrado por diferentes módulos comprendidos en una aplicación web y móvil los cuales se describen a continuación:

Módulo de Autenticación: Este módulo permitirá al usuario iniciar sesión, el tiempo de sesión y el cierre de sesión.

Módulo de Gestión de Usuarios: Este módulo permitirá al usuario crear una cuenta, cambiar las credenciales de su cuenta y eliminar la cuenta.

Módulo de gestión de clases:

Por parte del profesor: puede crear, eliminar y editar sus clases, anuncios y rubricas de evaluación

Por parte del alumno: unirse a una clase mediante un código alfanumérico generado aleatoriamente por el sistema y publicar un anuncio.

Módulo de gestión de Tareas:

Por parte del profesor: Crear, eliminar, visualizar archivos, calificar, devolver y comentar en privado por tarea y alumno.

Por parte del alumno: comentar en privado, subir archivo, eliminar el archivo, visualizar archivos, entregar y cancelar la entrega de tareas.

Módulo de Seguridad: La aplicación contempla algoritmos de cifrado hash para proteger datos sensibles, tales como contraseñas de usuarios, garantizando integridad más privacidad informativa dentro del entorno.

Limitaciones.

El desarrollo web deberá de ser operada en navegadores basados en la versión 146.0.7680.164 de Chromium, pues versiones anteriores podrían afectar el funcionamiento de la aplicación al no estar adaptada a ellas.

La aplicación móvil deberá de ser operada en Android 15 y versiones posteriores, ya que la aplicación puede presentar errores en sistemas anteriores por su configuración.

La aplicación no cuenta con edición colaborativa, videoconferencias ni conexión con otros programas, ya que se busca que el uso sea lo más sencillo posible.

Las notificaciones de la aplicación necesitan que el dispositivo tenga conexión a internet, ya sea por datos móviles o redes de banda ancha.

La carga de archivos tiene un límite de 50B por el servidor; si un archivo pesa más de esto, no se podrá subir.

Se podrán subir 5 archivos por asignación por el servidor; no se podrán subir más archivos una vez se llegue al límite.

La aplicación solo está disponible en el idioma español, no se podrá cambiar a otro(s) idioma(s) de preferencia del usuario.

La duración de los audios para la retroalimentación estará limitada a 1 minuto por comentario.

El límite para la detección de IA será de 20,000 caracteres por archivo de Microsoft 365.

Planeación estratégica.

-Ideario de la empresa.

-Misión.

Empoderar la superación personal y profesional a través de tecnología con sentido humano. En Uxmal, diseñamos soluciones simples e intuitivas que resuelven problemáticas específicas, facilitando que cada individuo elimine barreras y alcance sus objetivos. Creemos en el poder de los cambios paulatinos: transformamos el mundo de persona en persona, una solución a la vez.

-Visión.

Ser la empresa responsable de herramientas que impulsen la autonomía humana sin caer en la dependencia digital. Aspiramos a que Uxmal sea el motor de hábitos sustentables, donde el éxito no se mida por el tiempo en la pantalla, sino por la libertad y el crecimiento alcanzados fuera de ella.

-Logo.

El isotipo de Uxmal utiliza una geometría de líneas interconectadas que forman pirámides estilizadas. Esta estructura representa solidez y fundamentos firmes, aludiendo a la arquitectura de la ciudad antigua, pero bajo una estética de digital moderna.

En un entorno saturado de colores diseñados para generar dependencia, el uso de los colores blanco y negro, comunican transparencia y autoridad. Su contraste simboliza la claridad mental que la empresa busca devolver al usuario. Es una identidad atemporal que prioriza la legibilidad y la esencia del servicio sobre el ruido visual.

ILUSTRACIÓN 1 LOGO UXMAL



-Eslogan.

“Hecho para durar.”

Este eslogan funciona en dos niveles: el *físico* y el *humano*. Por un lado, hace referencia a las piedras de Uxmal, construcciones que han resistido siglos; por otro, habla de la creación de soluciones digitales que no son "modas" ni parches temporales. Significa que lo que Uxmal construye busca ser un pilar en la vida del usuario, una herramienta que no se vuelve obsoleta porque está diseñada con una base sólida.

En una era de software desechable y aplicaciones que duran un mes en el celular, *"Hecho para durar"* posiciona a la empresa como una firma de ingeniería seria que prioriza la calidad del código y una utilidad a largo plazo.

El eslogan refuerza la idea de que los cambios que el usuario logre con nuestras herramientas serán permanentes. No son una ayuda rápida, es una base que busca perdurar para siempre.

-Filosofía.

Creemos que la tecnología debe ser un puente hacia la autonomía, no una jaula de cristal. En un mundo saturado de algoritmos diseñados para retener la atención, Uxmal nace para devolverla.

Nuestra cultura rechaza la dependencia digital. Diseñamos bajo la premisa de que la mejor tecnología es la que se vuelve invisible: aparece cuando se necesita, resuelve el problema de forma simple y se retira para dejar que el individuo viva su realidad.

Fomentamos el cambio de "persona en persona". No buscamos transformaciones masivas y superficiales, sino victorias individuales y silenciosas que, acumuladas, generan un impacto real en la sociedad.

La arquitectura de Uxmal nos inspira a buscar la solidez en lo esencial. Si algo es complejo, no es útil. Nuestra ingeniería es transparente, honesta y está hecha para durar, eliminando cualquier truco psicológico que busque crear hábitos artificiales.

Buscamos que el usuario desarrolle sus propias capacidades. Nuestra meta final es que nuestras herramientas dejen de ser necesarias porque la persona ya ha integrado el hábito de superación en su propia naturaleza.

-Valores.

- **Autonomía.**

Diseñamos para que seas dueño de tus decisiones. Nuestra meta es que nuestras herramientas te impulsen a caminar por tu cuenta.

- **Sencillez.**

Eliminamos lo complejo. Si una función no simplifica la vida del usuario y no es fácil de usar, no tiene lugar en nuestra aplicación.

- **Honestidad.**

Sin trucos psicológicos ni notificaciones invasivas. Priorizamos la utilidad real sobre cualquier métrica de retención o dependencia.

- **Persistencia.**

Construimos resultados que permanecen. Valoramos la constancia y los hábitos pequeños que están hechos para durar toda la vida.

- **Respeto.**

Tu tiempo es sagrado. Trabajamos bajo la premisa de que la tecnología debe servir a la persona, nunca a la inversa.

-FODA.

TABLA 1 FODA

Fortalezas (Interno +)	Oportunidades (Externo +)
• Roles bien definidos: Cada integrante tiene una función clara en el equipo, lo que permite el avance técnico sin solapamiento de tareas. • Capacidad de trabajo autónomo: Aptitud para cumplir con entregables individuales de calidad de forma independiente. • Identidad de marca sólida: Contamos con una misión, visión y valores ya establecidos que guían el proyecto de una manera eficaz.	• Estandarización de procesos: Posibilidad de crear un flujo de trabajo que unifique los distintos enfoques de los integrantes. • Crecimiento del equipo: El tiempo de integración restante permite fortalecer la confianza a través de logros parciales. • Validación externa: El vacío informativo en plataformas como Teams y Classroom valida la relevancia de nuestro esfuerzo actual.
Debilidades (Interno -)	Amenazas (Externo -)
• Baja cohesión interna: Falta de comunicación y confianza debido al poco tiempo de integración del equipo. • Enfoques de trabajo heterogéneos: Diferencias en las metodologías individuales que pueden generar fricciones en la integración final. • Proyecto inconcluso: El software aún se encuentra en fase de desarrollo, lo que genera presión sobre los plazos de entrega.	• Riesgo de desalineación: Que los distintos enfoques de trabajo resulten en un producto final fragmentado o inconsistente. • Tiempos de entrega: La fecha límite de la tesina frente al estado actual del desarrollo de nuestro software.

Capítulo 2 Fundamento Teórico

Antecedentes

El presente apartado reúne investigaciones y proyectos relacionados con sistemas educativos y aplicaciones de gestión académica, con el propósito de proporcionar información y referencias que sirvan como base para el desarrollo de la presente tesina.

Reconocimiento automático del rostro para verificación de identidad para evaluación en línea

Creada por Wendy Valderrama Cárdenas en el Tecnológico Nacional de México (CENIDET) en 2019.

El sistema permite validar la identidad del alumno durante exámenes en línea usando reconocimiento facial. Captura imágenes desde la cámara web, compara rostros automáticamente y monitorea al usuario durante toda la evaluación para detectar cambios de identidad o ausencia frente a la cámara.

Diseño, desarrollo e implementación de una plataforma educativa

Creada por Ixban Isaí López Ibañez en la UNAM en 2019.

La plataforma permite registrar usuarios, administrar cursos, publicar materiales académicos y consultar contenidos desde navegador web. Incluye módulos para alumnos y docentes, control de acceso y gestión de información académica.

Desarrollo de una plataforma educativa para dispositivos móviles

Creada por Ángel Izquierdo Ayala y Gustavo López Paniagua en la UNAM en 2023.

La aplicación móvil permite consultar materiales educativos, acceder a contenidos académicos y visualizar información escolar desde teléfonos móviles. Incluye navegación entre módulos educativos y acceso remoto para estudiantes.

Diseño de una plataforma web educativa para asesorías y consulta de materiales de apoyo

Creada por David Alejandro Hernández Garduño en la UNAM en 2022.

El sistema permite agendar asesorías, consultar materiales académicos y administrar recursos educativos desde una plataforma web. Incluye gestión de usuarios, almacenamiento de archivos y acceso a contenidos de apoyo escolar.

Construcción de aplicación web para la automatización de una evaluación de regulación emocional dirigida a adolescentes

Creada por Oscar Duarte Morales en la UNAM en 2017.

La aplicación web permite aplicar evaluaciones digitales, registrar respuestas automáticamente y generar resultados de forma inmediata. Incluye formularios interactivos, almacenamiento de respuestas y visualización de resultados.

Evaluación de una plataforma educativa para el aprendizaje reflexivo

Creada por Ámbar Enith Gómez Cuestas en la UNAM en 2022.

La plataforma permite acceder a actividades educativas, consultar contenidos y realizar ejercicios de aprendizaje reflexivo. Incluye herramientas de interacción entre usuarios y evaluación de experiencia de uso.

Determinación de Servicios Web de Aprendizaje con base en un Modelo de Procesos

Creada por Vitervo López Caballero en el TecNM en 2021.

El sistema genera servicios web educativos para administrar actividades, contenidos y procesos de aprendizaje. Permite integrar recursos académicos y automatizar tareas relacionadas con plataformas educativas.

Uso de Reglas de Dominio para Evaluar Recursos de Servicios Web de Aprendizaje

Creada por José Antonio Sandoval Acosta en el TecNM en 2023.

El sistema evalúa recursos educativos digitales mediante reglas automáticas. Permite analizar materiales académicos, validar información y medir calidad de contenidos educativos.

Generación de SWA a partir de Objetos de Aprendizaje

Creada por Enrique Jonathan Cordero Escobar en el TecNM en 2023.

El sistema transforma objetos de aprendizaje en servicios web reutilizables para plataformas educativas. Permite integrar actividades, contenidos y recursos académicos dentro de aplicaciones educativas.

Marco de Servicios Web para la Medición de Relevancia en Recursos de Aprendizaje

Creada por Juan Francisco Morales Martínez en el TecNM en 2024.

El sistema analiza recursos educativos digitales y calcula su relevancia mediante procesamiento automático de información. Incluye análisis de contenidos, comparación de textos y generación de métricas.

Definición de términos básicos.

Glosario de términos.

El presente glosario reúne los términos técnicos y conceptuales empleados a lo largo de este documento, con el propósito de unificar el lenguaje utilizado y facilitar la comprensión del lector. Los conceptos se presentan en orden alfabético y corresponden tanto a elementos educativos como a componentes tecnológicos propios de la aplicación MaxGrade.

Administrador: Perfil con privilegios elevados para gestionar usuarios, roles y configuraciones globales.

Alumno: Usuario registrado en la plataforma con permisos para entregar tareas y archivos.

Credenciales: Datos utilizados para autenticar a un usuario en el sistema.

Cuenta: Registro digital que contiene la información necesaria para identificar y autenticar a un usuario.

Equipo: Subconjunto de usuarios pertenecientes a un grupo.

Evidencia: Archivo adjunto a una tarea como soporte o comprobación de su realización.

Grupo: Conjunto de usuarios asociados para la gestión de actividades académicas.

Notificación: Alerta generada por el sistema en función de la configuración del alumno.

Profesor: Usuario registrado en la plataforma con permisos para evaluar tareas y gestionar clases y tareas.

Registro: Acción de almacenar información en la base de datos del sistema.

Rúbrica: Instrumento de evaluación formal que establece criterios específicos y niveles de calidad para juzgar un desempeño o producto.

Sesión: Estado activo que permite a un usuario autenticado interactuar con funcionalidades restringidas.

Soporte: Servicio o entidad encargada de resolver incidencias técnicas y asistir a los usuarios.

Tarea: Actividad académica registrada en el sistema.

Definición Operacional

Unidad Docente

La figura del docente es central en cualquier sistema de gestión educativa, ya que es quien organiza las actividades, evalúa el desempeño de los estudiantes y guía el proceso de aprendizaje. En esta unidad se exploran las distintas dimensiones del rol docente: quién es, qué tipos existen, cómo enseña, qué necesita y a qué desafíos se enfrenta; elementos que fundamentaron el diseño de las funcionalidades orientadas al profesor dentro de la aplicación MaxGrade.

- Tipos de docente

De acuerdo con la Universidad Madero (s.f) existen 5 tipos de docentes, los cuales son:

TABLA 2 TIPOS DE DOCENTE

Tipo de Docente	Descripción
Tradicionalista	Se caracteriza por ser el transmisor activo de los conocimientos utilizando métodos tradicionales o clásicos de la enseñanza, como pueden ser la lección magistral, el dictado, lectura y repetición mecánica de conceptos, los alumnos son en su mayoría pasivos siendo únicamente receptores de información.
Innovador	Se caracterizan por ser flexibles y poder adaptarse a las necesidades de los alumnos o al entorno del contexto actual, utilizando distintas metodologías para fomentar el aprendizaje.
Facilitador	Se caracteriza por ser un guía que facilita el acceso al conocimiento en lugar de solo transmitirlo a los alumnos, fomentando el aprendizaje autónomo y el pensamiento crítico en los alumnos.
Investigador	Se caracteriza por combinar la enseñanza y cuestionar su forma de enseñar para encontrar problemas en su ambiente educativo, buscando mejorar constantemente su práctica y enseñanza.

Orientador	Se caracteriza por no solo enseñar, sino que también por apoyar a los alumnos en su desarrollo personal y profesional, brindando apoyo para identificar sus intereses, fortalezas y metas académicas.
------------	---

Esta diversidad de perfiles docentes fue un referente importante en el diseño de MaxGrade, ya que la aplicación debía ser lo suficientemente flexible para adaptarse a diferentes estilos de trabajo. Por ello, a continuación, se analiza precisamente esa variedad de estilos de enseñanza.

- Estilo de enseñanza del docente:

Se define como la manera en que el profesor organiza su labor y se relaciona con sus alumnos, basada en el modelo de Portilho y Banas (Perochena et al., 2017), la enseñanza se clasifica en cuatro tipos:

TABLA 3 ESTILO DE ENSEÑANZA DEL DOCENTE

Estilo de Enseñanza	Descripción
Dinámico	Lo abarcan profesores que se enfocan más en la interacción y el aprendizaje a través del debate, dando lugar a la exploración de diversos materiales y el planteamiento de preguntas abiertas.
Analítico	Lo abarcan docentes que se centran en el temario dado y en la planificación detallada, su objetivo principal es asegurar que el alumno comprenda cada tema, dando el tiempo suficiente para el análisis, repaso y reflexión.
Sistemático	Lo abarcan docentes que buscan el orden y una lógica estructural en la enseñanza, fomentan el aprendizaje autónomo y el intercambio de ideas exigiendo que las ideas dadas estén respaldadas por argumentos sólidos.
Practico	Lo abarcan docentes que buscan la aplicación útil del conocimiento, basándose en la experimentación y desarrollo de soluciones a problemas reales en situaciones cotidianas del entorno del estudiante.

Independientemente del estilo de enseñanza que adopte, el ejercicio docente está también condicionado por una serie de necesidades institucionales y personales. Comprender dichas necesidades fue esencial para identificar qué herramientas debería proveer la plataforma.

- Necesidades del docente:

Boluda (2015) basándose en la teoría de motivación humana de Maslow propone una jerarquía de necesidades para los docentes que se estructuran en 5 niveles ascendentes:

TABLA 4 NECESIDADES DEL DOCENTE

Estabilidad	Se refiere a las condiciones laborales básicas como un sueldo justo y un horario adecuado para la planeación de la clase.
Formación	Se refiere a la preparación pedagógica y la actualización constante en herramientas digitales y metodologías del contexto actual.
Recursos	Se refiere a la disponibilidad de la infraestructura como puede ser servicios de internet o mobiliario adecuado
Reconocimiento	Se refiere al apoyo y validación del trabajo por parte de la comunidad
Autorrealización	El punto máximo del ejercicio docente, donde la vocación pasa a ser una práctica que genera un impacto positivo.

Satisfacer estas necesidades no siempre resulta sencillo. El entorno educativo actual plantea obstáculos concretos que el docente debe afrontar de forma cotidiana, y cuyo reconocimiento resulta igualmente relevante para este proyecto.

- Desafíos que enfrentan los docentes:

Los docentes se enfrentan al desafío de integrar la tecnología de manera efectiva en su enseñanza, utilizando herramientas digitales y plataformas educativas para enriquecer la experiencia de aprendizaje; sin embargo, este proceso también requiere habilidades técnicas y pedagógicas para asegurar que los estudiantes se beneficien de dichas herramientas sin perder el enfoque educativo. Uxmal surge precisamente como respuesta a esta necesidad: una plataforma diseñada para reducir la curva de adopción

tecnológica del docente, centralizando en un solo sistema la creación de tareas, la revisión de entregas y el seguimiento del rendimiento académico, de modo que el profesor pueda concentrar su energía en lo que realmente importa: enseñar.

Unidad Experiencia de Usuario

El diseño centrado en la experiencia del usuario (UX) determina en gran medida si una plataforma digital es adoptada con éxito o abandonada por sus usuarios. En esta unidad se exponen los conceptos fundamentales de UX y los elementos de interfaz que guiaron el diseño visual e interactivo de Uxmal, con el objetivo de garantizar una experiencia intuitiva tanto para docentes como para alumnos.

- ¿Qué es UX?:

La experiencia de usuario se refiere a la impresión que adquiere el usuario al interactuar con un producto, esta tiene como objetivo satisfacer las necesidades generales del usuario, algunos de los parámetros que componen la UX son:

TABLA 5 ELEMENTOS DE UX

Diseño Intuitivo	Es la práctica de diseñar, pantallas, botones, formularios e interfaces que den a entender su función de manera clara al usuario. Se basa en la coherencia, la simplicidad y en anticipar las necesidades de usuario, utilizando sus conocimientos previos
Navegación de usuario	Se refiere a la interacción que tiene el usuario al momento de moverse dentro del sistema, utilizando herramientas como hipervínculos, menús o demás funciones para facilitar la navegación del usuario dentro del sistema
Flujo de Usuario	Es un diagrama que representa paso a paso la interacción que debería tener un usuario con el sistema para cumplir el propósito final del sistema
Elementos de UI	Son los componentes que conforman en su totalidad a la UI (Interfaz de usuario). Ejemplos clave son botones, campos de texto, menús desplegables, íconos, barras de navegación y elementos informativos. Se clasifican comúnmente en entrada, navegación e informativos

Interacción Humano-Computadora	Es una ciencia multidisciplinaria que se encarga del estudio, el diseño y la evaluación de los humanos interactuando con los sistemas digitales en las computadoras. Busca crear tecnologías que sean eficientes, seguras, fáciles de usar y efectivas, mejorando la experiencia del usuario (UX) mediante interfaces intuitivas
--------------------------------	--

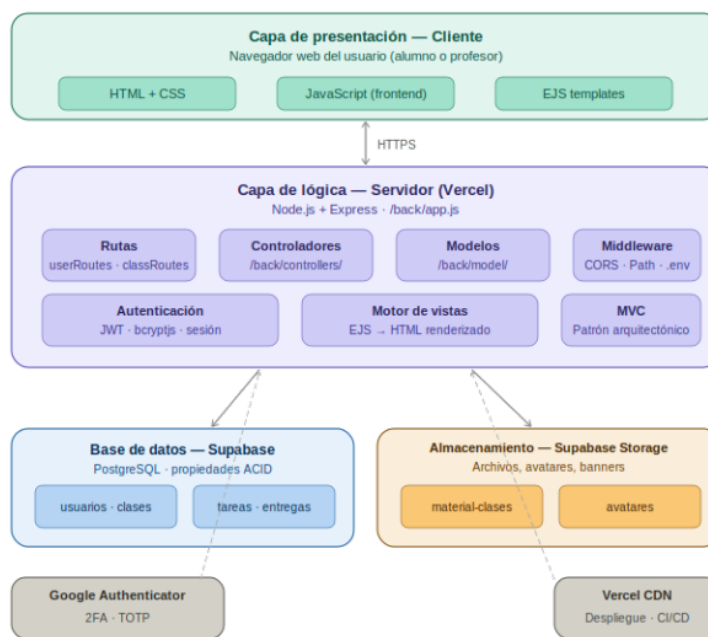
Unidad Arquitectura del software

Las decisiones de arquitectura y tecnología sobre las que se construye un sistema determinan su rendimiento, mantenibilidad y capacidad de crecer en el tiempo. En esta unidad se presentan los fundamentos teóricos de cada componente tecnológico utilizado en MaxGrade desde el servidor y los patrones de diseño hasta los lenguajes, frameworks y servicios en la nube, a su vez se explica en cada caso la razón por la que fue seleccionado y cómo se integra en el conjunto del sistema.

- ¿Qué es un servidor?:

Un servidor es una instancia de una computadora o un programa informático que proporciona servicios, recursos o datos a otros programas o dispositivos denominados clientes a través de una red local o internet. Su función principal es atender peticiones, procesar información y devolver una respuesta precisa, operando de manera continua para garantizar la disponibilidad de los recursos. Estos sistemas cuentan con hardware especializado o entornos virtuales diseñados para gestionar cargas de trabajo constantes, asegurando que la integridad de los datos y la seguridad de las transacciones se mantengan bajo estándares técnicos rigurosos. Nuestro servidor está representado mediante el siguiente diagrama:

ILUSTRACIÓN 2 DIAGRAMA DE ARQUITECTURA DEL SISTEMA



- Tipos de servidores:

Los servidores se categorizan según el propósito específico para el que han sido configurados, permitiendo una especialización que optimiza el rendimiento general de una red de computadoras. Entre los más comunes se encuentran los servidores web, que alojan sitios; los de bases de datos, especializados en el almacenamiento y consulta de registros; y los de aplicaciones, que ejecutan la lógica de negocio compleja. La elección del tipo de servidor depende directamente de la carga de tráfico esperada, la naturaleza de los datos procesados y la arquitectura de software que se haya decidido implementar para la solución tecnológica.

En MaxGrade utilizamos principalmente un servidor web con Express que sirve contenido HTTP y responde a solicitudes GET, POST, PUT y DELETE. Express actúa también como servidor de aplicaciones al ejecutar la lógica de autenticación, validación de tareas, cálculo de calificaciones y gestión de entregas. Este rol dual servidor web y de aplicaciones simplificó la arquitectura y redujo la latencia al eliminar capas intermedias innecesarias.

Para que ese servidor opere de manera coherente y sostenible, es fundamental que su código esté organizado siguiendo principios arquitectónicos sólidos. Uno de los patrones más utilizados en el desarrollo web para lograr esa organización es el Modelo-Vista-Controlador.

- Patrones de diseño:

Los patrones de diseño son soluciones probadas y documentadas para problemas recurrentes en el diseño de software, actuando como plantillas que facilitan la creación de código limpio y eficiente. No son fragmentos de código terminados que se pueden copiar y pegar, sino conceptos generales que ayudan a los desarrolladores a estructurar sus sistemas de una manera que sea fácil de mantener y escalar. Al utilizar estos estándares, se reduce la complejidad del desarrollo, se minimizan los errores lógicos y se mejora la comunicación técnica entre los miembros de un equipo de programación al hablar un lenguaje común.

- Patrón Modelo Vista-Controlador (MVC)

El patrón Modelo-Vista-Controlador (MVC) es un patrón arquitectónico de software que divide una aplicación en tres componentes interconectados, con el objetivo de separar la lógica interna del programa de su representación visual y de la gestión de las interacciones del usuario. Esta separación facilita el mantenimiento, las pruebas y la escalabilidad de la aplicación.

Los tres componentes del patrón son: el Modelo, que representa los datos y la lógica de negocio, encargándose de acceder y manipular la información almacenada en la base de datos; la Vista, que es la capa de presentación visible para el usuario, responsable de mostrar la información de manera comprensible; y el Controlador, que actúa como intermediario entre el Modelo y la Vista, recibiendo las acciones del usuario, procesando la lógica correspondiente y actualizando tanto el modelo como la vista según sea necesario.

En MaxGrade, la implementación de MVC es directa y refleja esa separación en la estructura de carpetas del proyecto: los modelos en `"/back/model/"` contienen las consultas a la base de datos PostgreSQL a través de Supabase; los controladores en `"/back/controllers/"` contienen la lógica que decide qué datos recuperar, cómo procesarlos y qué vista renderizar; y las vistas en `"/front/views/"` son las plantillas EJS que generan el HTML enviado al navegador. Esta organización permitió que la lógica de negocio, la presentación y el acceso a datos se pudieran modificar de forma independiente sin afectar al resto del sistema.

- Arquitecturas de Servidores:

La arquitectura de servidores se refiere al diseño y la disposición lógica de los componentes tecnológicos que conforman un sistema de servicios en red, determinando cómo interactúan entre sí. Este diseño establece las bases de la

escalabilidad, la tolerancia a fallos y la eficiencia operativa, definiendo si el procesamiento se centraliza en un solo lugar o se distribuye entre múltiples nodos. Una arquitectura bien planificada es crucial para prevenir cuellos de botella y asegurar que los recursos del hardware se utilicen de manera óptima para responder a las demandas de los usuarios finales en todo momento.

Utilizamos una arquitectura de tres capas: la capa de presentación (frontend con HTML, CSS y JavaScript en `"/front/"`), la capa de lógica de aplicación (servidor Express en `"/back/"` con controladores y rutas), y la capa de datos (PostgreSQL en Supabase). Cada capa es independiente y puede desarrollarse, probarse y escalarse por separado. Aplicamos una arquitectura monolítica donde todo el código backend está en un único servidor Express, aunque está bien estructurado para migrar a microservicios si es necesario. Utilizamos arquitectura cliente-servidor, donde el navegador hace solicitudes HTTP al servidor que procesa y devuelve respuestas. Implementamos un modelo principalmente stateless, aunque usamos sesiones para casos específicos como mantener la sesión del usuario autenticado.

- Tipos de Arquitecturas de Servidores:

Existen diversos modelos arquitectónicos que definen la jerarquía y la forma en que los sistemas se comunican, como la arquitectura monolítica, la de microservicios o el modelo cliente-servidor tradicional. Cada tipo ofrece ventajas distintas: mientras que una arquitectura monolítica es más simple de desplegar inicialmente, los microservicios permiten una escalabilidad independiente de cada función. La elección del modelo adecuado es una decisión estratégica que impacta directamente en la velocidad de desarrollo, el costo de los servidores y la facilidad con la que se pueden implementar nuevas características técnicas en el futuro.

Utilizamos una arquitectura monolítica donde toda la lógica de la aplicación se concentra en un único servidor Express. Esto facilita el desarrollo inicial y despliegue simple. El código está estructurado de manera que podría migrarse a arquitectura de microservicios en el futuro, separando por ejemplo la autenticación, gestión de clases, y evaluación en servicios distintos. Implementamos parcialmente una arquitectura serverless con Supabase como backend-as-a-service, que abstrae la complejidad de infraestructura de base de datos y almacenamiento. Express actúa como nuestro único punto de entrada (API Gateway) que gestiona todas las solicitudes del cliente.

- Lenguaje de Programación:

Un lenguaje de programación es un sistema formal diseñado para que los seres humanos puedan comunicar instrucciones precisas a una computadora, permitiendo la

creación de software complejo. Estos lenguajes poseen reglas gramaticales y sintácticas propias que deben seguirse estrictamente para que el procesador pueda interpretar y ejecutar las tareas deseadas. A través de ellos, los desarrolladores pueden manipular datos, controlar periféricos y crear algoritmos que resuelven problemas específicos, transformando ideas abstractas en aplicaciones funcionales que impactan en la vida cotidiana de las personas y las organizaciones.

En MaxGrade utilizamos JavaScript tanto en backend como en frontend, proporcionando una ventaja significativa: los desarrolladores trabajan con un único lenguaje en toda la pila tecnológica. Permitiendo reutilizar conceptos y patrones. Utilizamos JavaScript moderno (ES6 y posteriores) con características como funciones flecha, deestructuración, `async/await` y promesas que hacen el código más limpio y legible.

- JavaScript:

JavaScript es un lenguaje de programación versátil y de alto nivel que se ha convertido en el estándar para el desarrollo de aplicaciones web interactivas y dinámicas. Su capacidad para ejecutarse tanto en el navegador del cliente como en el entorno del servidor (mediante Node.js) lo hace extremadamente eficiente para proyectos modernos de pila completa. Es un lenguaje orientado a objetos y basado en prototipos que permite actualizaciones de contenido en tiempo real, control de multimedia y animaciones complejas, siendo una pieza esencial para cualquier proyecto digital que busque una interacción directa con el usuario.

En backend con Node.js, escribimos el servidor Express que maneja operaciones de lógica de negocio, consultas a base de datos, autenticación y gestión de sesiones. En los controladores (`/back/controllers/`), utilizamos JavaScript asíncrono con `async/await` para manejar operaciones que toman tiempo como consultas a la base de datos o carga de archivos. En frontend (`/front/public/js/`), utilizamos JavaScript para realizar operaciones asíncronas con el servidor (mediante `fetch API`), manipular el DOM para actualizar la interfaz del usuario, validar datos antes de enviarlos al servidor, enviar solicitudes POST, PUT y DELETE a endpoints API, y procesar respuestas dinámicamente sin recargar la página. Implementamos validaciones complejas tanto del lado del cliente como del servidor.

- Base de datos:

Forma de almacenar datos de forma estructurada para gestionar, recuperar y actualizar datos de manera eficiente. Hacen uso de sistemas de gestión para manejar grandes

volúmenes de datos, los más comunes usados hoy en día son relacionales y no relacionales.

Utilizamos PostgreSQL a través de Supabase como base de datos relacional que almacena toda la información de la aplicación: usuarios con sus roles (profesor o estudiante), clases, tareas, entregas, rúbricas de evaluación y publicaciones. La base de datos está estructurada en tablas como usuarios, clases, tareas, entregas, rúbricas y publicaciones, con relaciones bien definidas entre ellas. Supabase actúa como una capa de abstracción que facilita el acceso a PostgreSQL sin gestionar infraestructura directamente. Implementamos validaciones en el servidor, utilizamos consultas parametrizadas para evitar inyecciones SQL, y verificamos permisos mediante middleware de autenticación para garantizar la seguridad de los datos.

- Motor de BD

Utilizamos PostgreSQL como nuestro sistema de base de datos relacional alojado en Supabase. Implementamos características como vistas, índices, y restricciones de integridad referencial. PostgreSQL garantiza propiedades ACID: atomicidad (las operaciones se completan totalmente o no se ejecutan), consistencia (datos siempre en estado válido), aislamiento (transacciones no interfieren entre sí) y durabilidad (datos persistidos no se pierden). En Uxmal, PostgreSQL gestiona toda la información sobre usuarios, clases, tareas, entregas y evaluaciones. Accedemos a PostgreSQL sin gestionar el servidor directamente mediante Supabase. Las tablas incluyen restricciones como claves primarias, claves foráneas y validaciones CHECK para garantizar integridad de datos. Realizamos consultas complejas con JOINS, subconsultas y funciones de agregación para obtener información consolidada como desempeño de estudiantes o estadísticas de entregas.

- HTML:

HTML (HyperText Markup Language por sus siglas en inglés) es el código estándar más comúnmente usado para estructurar y desplegar páginas web. Funciona mediante etiquetas usadas para definir los elementos de una página web como lo son: `<h1>` para títulos, `<p>` para párrafos, `<img>` para imágenes, `<video>` para los videos, `<a>` para enlaces, entre otras. Funciona como el “esqueleto” funcional de cualquier página o sitio web. Es un lenguaje de marcado utilizado para organizar el contenido, no un lenguaje de programación con lógica.

Utilizamos HTML en múltiples vistas ubicadas en `/front/views/`, incluyendo `index.html` (página de inicio), `login.ejs`, `register.ejs`, `dashboard.ejs` (panel principal), `clase.ejs` (visualización de clases), `tareas.ejs` (lista de tareas), `detalle_tarea.ejs` (detalles de

task), rendimiento.ejs (estadísticas del estudiante), y perfil.ejs (información del usuario). Cada archivo contiene etiquetas HTML semánticas como <header>, <nav>, <main>, <section>, <article>, <form> y <footer> que estructuran el contenido. Los formularios HTML incluyen campos de entrada (<input>), áreas de texto (<textarea>), selecciones (<select>), y botones de envío que permiten a los usuarios interactuar con la aplicación. Utilizamos validación nativa de HTML5 como required, pattern, min, max, type="email" y type="password" para mejorar la experiencia del usuario y realizar validaciones básicas del lado del cliente

- CSS:

CSS (Cascading Style Sheets por sus siglas en inglés) es uno de los lenguajes de diseño más utilizados hoy en día para definir la presentación visual de documentos estructurados con HTML o XML. Permite separar el contenido de la apariencia, facilitando el control de colores, fuentes, y maquetación de sitios web de una forma sencilla.

Utilizamos archivos CSS externos ubicados en /front/public/css/: style.css (estilos generales), `auth.css` (estilos para login y registro), dashboard.css (estilos del panel principal), e index.css (estilos de inicio). Estos archivos definen el esquema de colores, el diseño con CSS Grid y Flexbox, tipografía, espaciado consistente, y estilos de botones, formularios y tarjetas. Implementamos diseño responsivo mediante media queries para que la aplicación se vea bien en teléfonos móviles, tablets y computadoras de escritorio. Utilizamos CSS moderno incluyendo variables CSS (custom properties) para mantener un esquema de colores consistente, transiciones suaves para mejorar la experiencia del usuario, y efectos visuales como sombras y bordes redondeados que hacen la interfaz más atractiva y profesional.

- Node:

Node.js es un entorno de ejecución para JavaScript construido con el motor de búsqueda V8 de Chrome, que permite ejecutar código de programación fuera del navegador, específicamente en el lado del servidor. Su arquitectura está orientada a eventos y es de entrada/salida sin bloqueo, lo que lo hace extremadamente ligero y eficiente para manejar aplicaciones que requieren el procesamiento de datos de forma intensiva y en tiempo real. Gracias a su amplio ecosistema de paquetes, permite a los desarrolladores construir herramientas de backend escalables utilizando el mismo lenguaje que se emplea habitualmente en el desarrollo de interfaces de usuario para la web.

Node.js es el entorno donde ejecutamos nuestro servidor Express. El archivo "back/app.js" contiene la configuración del servidor que se ejecuta en Node.js, escuchando en un puerto específico (típicamente 3000) y respondiendo a solicitudes HTTP. El archivo "back/package.json" especifica todas las dependencias del proyecto que Node.js descarga e instala mediante npm. Node.js permite que nuestro equipo utilice JavaScript en ambos lados de la aplicación (cliente y servidor), reduciendo la curva de aprendizaje y permitiendo reutilizar código y patrones. Node.js proporciona módulos nativos que utilizamos en nuestro código: el módulo path para trabajar con rutas de archivos, fs para acceso al sistema de archivos, http para crear servidores, y muchas otras funcionalidades. El modelo de entrada/salida no bloqueante y orientado a eventos de Node.js permite que nuestro servidor maneje miles de conexiones simultáneas de manera eficiente.

- Express:

Express es un marco de trabajo (framework) para aplicaciones web minimalista y flexible que se ejecuta sobre Node.js, diseñado para facilitar la creación de aplicaciones robustas y APIs de forma sencilla. Proporciona un conjunto robusto de características para el desarrollo web, como el manejo de rutas, la gestión de sesiones y la integración de diferentes motores de plantillas, sin ocultar las características principales de Node.js. Es ampliamente considerado el estándar de facto para la creación de servicios backend en la industria debido a su velocidad, modularidad y la gran comunidad de soporte que permite resolver problemas técnicos complejos rápidamente.

Express es el framework que utilizamos en back/app.js para configurar el servidor backend. Define todas las rutas de la aplicación organizadas en archivos separados: userRoutes.js (autenticación, registro, actualización de perfil), classRoutes.js (operaciones sobre clases), assignmentRoutes.js (operaciones sobre tareas), unitsRoutes.js (unidades de aprendizaje), rubricRoutes.js (rúbricas de evaluación), y postRoutes.js (publicaciones o foros). Cada ruta está conectada a controladores que manejan la lógica de negocio. Express permite definir middleware personalizado que se ejecuta antes de que una solicitud llegue al controlador final; utilizamos middleware para autenticación, verificación de roles, parsing de JSON, manejo de cookies, y sesiones. Configuramos el motor de vistas EJS, servimos archivos estáticos (CSS, JS, imágenes), y definimos todos los endpoints que el frontend consume mediante solicitudes HTTP.

- Cors:

CORS (Cross – Origin Resource Sharing) es un mecanismo de seguridad basado en cabeceras HTTP que permite a un servidor indicar cualquier origen (dominio, esquema o puerto) distinto al suyo desde el cual un navegador debería permitir la carga de recursos. Es fundamental en la web moderna para prevenir ataques maliciosos, asegurando que solo los sitios web autorizados puedan realizar peticiones a una API privada. Sin una configuración adecuada de CORS, los navegadores bloquean por defecto cualquier intento de comunicación entre dominios diferentes, lo que protege la integridad de los datos y evita que aplicaciones externas no verificadas extraigan información sensible de los servidores.

Implementamos CORS en `back/app.js` con la línea `app.use(cors())`, que habilita CORS para todas las solicitudes entrantes con configuración por defecto. Esto permite que el frontend (que está en <http://localhost:3001> o un dominio diferente durante desarrollo) pueda realizar solicitudes HTTP (GET, POST, PUT, DELETE) al backend (que se ejecuta en <http://localhost:3000>). Sin CORS configurado, el navegador bloquearía estas solicitudes con un error de CORS. La librería `cors` que importamos simplifica la configuración en Express, permitiendo establecer políticas más granulares si es necesario (como permitir solo ciertos orígenes, métodos específicos, o headers particulares). CORS es esencial para que la comunicación entre frontend y backend funcione correctamente.

- Path:

Path es un módulo nativo de Node.js que proporciona utilidades para trabajar con rutas de archivos y directorios de una manera compatible con diferentes sistemas operativos como Windows, Linux o macOS. Dado que la estructura de las carpetas varía entre sistemas (usando barras inclinadas o invertidas), este módulo resuelve automáticamente las discrepancias para evitar errores de ejecución en el servidor. Es una herramienta esencial para la manipulación de archivos, ya que permite unir rutas, extraer extensiones de documentos y normalizar direcciones de almacenamiento de forma segura y predecible durante el despliegue de cualquier software profesional.

Utilizamos el módulo `path` de Node.js en múltiples lugares del backend. En `back/app.js`, usamos `path.join(__dirname, ../front/views)` para indicar al motor de vistas EJS dónde se encuentran los archivos de plantilla, asegurándonos de que funcione correctamente sin importar desde dónde se ejecute la aplicación. También utilizamos Path para servir archivos estáticos: `app.use(/public, express.static(path.join(__dirname, ../front/public)))` configura Express para servir

todos los archivos CSS, JavaScript e imágenes del frontend desde la ruta correcta. En `UserController.js`, utilizamos `Path` para manejar rutas de archivos al procesar carga de imágenes de perfil. La función `path.join()` es particularmente útil porque resuelve referencias como `“..”` (directorio padre) de manera correcta. En `back/config/db.js`, también utilizamos `Path` para cargar el archivo `.env` de forma portátil.

- Variables de entorno (`.env`):

Las variables de entorno son valores externos a la aplicación que se utilizan para configurar el comportamiento del software sin necesidad de modificar el código fuente, protegiendo datos sensibles como claves de API o contraseñas. Se almacenan generalmente en un archivo oculto llamado `“.env”` y permiten que una misma aplicación se comporte de manera distinta según el entorno en el que se encuentre, ya sea desarrollo, pruebas o producción. Esta práctica es un estándar de seguridad crítico, ya que evita que información confidencial sea expuesta en repositorios de código públicos y facilita la gestión de credenciales en equipos de trabajo colaborativos.

Utilizamos `dotenv` requiriendo `“require('dotenv').config()”` en `“back/app.js”` y `“back/config/db.js”`. Esto carga automáticamente un archivo `“.env”` (que no debe estar versionado en Git por seguridad) que contiene variables como las credenciales de Supabase (`“SUPABASE_URL”` y `“SUPABASE_KEY”`), la configuración del puerto del servidor, la clave secreta para JWT, y otras configuraciones específicas del entorno. En `“back/config/db.js”`, leemos estas variables con `“process.env.SUPABASE_URL”` y `“process.env.SUPABASE_KEY”` para configurar la conexión a la base de datos. Las variables de entorno permiten que la misma aplicación funcione en diferentes entornos (desarrollo, pruebas, producción) con diferentes configuraciones sin modificar el código. Esta práctica es fundamental en seguridad, ya que evita que datos sensibles como claves de API se commiten al repositorio Git, reduciendo significativamente el riesgo de exposición de credenciales.

- EJS

EJS es un motor de plantillas simple para JavaScript que permite generar marcado HTML con código JavaScript plano de manera dinámica y eficiente en el lado del servidor. A diferencia de otros motores más complejos, EJS destaca por su sintaxis familiar que permite inyectar datos directamente en las etiquetas HTML, lo que facilita la creación de páginas web que cambian según la información recibida de una base de datos. Es ideal para aplicaciones que necesitan renderizar contenido personalizado rápidamente, manteniendo una separación clara entre la estructura visual del sitio y la lógica de programación que alimenta dicha interfaz.

Se configura en `back/app.js` con las líneas `app.set('view engine', 'ejs')` y `app.set('views', path.join(__dirname, '../front/views'))`, indicando a Express que utilice EJS como motor de plantillas. Aunque la aplicación está estructurada principalmente como una API REST que devuelve JSON, utilizamos EJS para renderizar la página de inicio y algunas vistas iniciales como login y registro. En vistas como `login.ejs`, podemos incluir variables dinámicas, URLs basadas en configuración, o condicionales que verifiquen si el usuario está autenticado. EJS facilita una transición suave entre vistas estáticas y dinámicas. Los parciales de EJS (mediante `<%- include() %>`) permiten dividir las plantillas en componentes reutilizables, mejorando la mantenibilidad del código. Esto es especialmente útil para elementos comunes como headers, footers, barras de navegación que se reutilizan en múltiples páginas.

- Bcryptjs:

bcryptjs es una biblioteca optimizada para Node.js que implementa un algoritmo de hashing de contraseñas altamente seguro, diseñado para proteger las credenciales de los usuarios contra ataques de fuerza bruta. A diferencia de otros métodos de encriptación simples, bcrypt incorpora una técnica llamada "salting" que añade datos aleatorios a cada contraseña antes de procesarla, generando un código único e irreversible para cada usuario. Es una herramienta estándar en la industria de la ciberseguridad, ya que su capacidad para ajustar el costo computacional del hash permite que el sistema siga siendo seguro incluso frente al aumento de la potencia de procesamiento de las computadoras actuales.

- Servidor App: Vercel

Vercel es una plataforma en la nube diseñada para el despliegue de aplicaciones web y funciones de servidor sin necesidad de gestionar infraestructura física, optimizada para ofrecer un rendimiento excepcional y tiempos de carga mínimos. Permite la integración continua y el despliegue automático, lo que significa que cualquier actualización en el código se refleja instantáneamente en el sitio en vivo a través de su red global de servidores (Edge Network). Su enfoque en la experiencia del desarrollador y su capacidad para escalar automáticamente la carga de tráfico la convierten en una de las opciones más eficientes para alojar aplicaciones modernas que requieren alta disponibilidad.

- Servidor BD: Supabase

Supabase es una plataforma de backend como servicio (BaaS) de código abierto que proporciona una base de datos PostgreSQL completa, junto con herramientas integradas de autenticación, almacenamiento de archivos y APIs en tiempo real. Se

destaca por ofrecer la potencia de las bases de datos relacionales tradicionales con la facilidad de uso de las tecnologías modernas en la nube, permitiendo a los desarrolladores gestionar esquemas de datos complejos de forma visual. Su arquitectura está diseñada para ser escalable y segura, proporcionando funcionalidades avanzadas como la escucha de cambios en vivo, lo que facilita la creación de aplicaciones altamente interactivas y colaborativas.

Supabase es el corazón de nuestra infraestructura de datos. En `“back/config/db.js”`, nos conectamos a Supabase usando la URL del proyecto y una clave pública. Todos nuestros modelos en `“/back/model/”` utilizan el cliente de Supabase para realizar operaciones CRUD sobre las tablas (usuarios, clases, tareas, entregas, rúbricas). Por ejemplo, en `“userModel.js”`, la función `“create()”` utiliza `“supabase.from('usuarios').insert([{{...}}])”` para insertar un nuevo usuario. En `“authController.js”`, usamos `“supabase.from('usuarios').select('*').eq('email', email).single()”` para buscar usuarios por email. Utilizamos también el servicio de almacenamiento de Supabase para guardar avatares mediante `“supabase.storage.from('avatares').upload()”`. Supabase simplifica significativamente el desarrollo al proporcionar una base de datos PostgreSQL completa sin necesidad de gestionar servidores, bases de datos, o respaldos manualmente, permitiendo que un equipo pequeño mantenga una aplicación escalable y confiable.

Capítulo 3 “Metodología de Desarrollo”

Viabilidad

Esta aplicación minimalista y accesible para docentes de 40 a 60 años busca mejorar la eficiencia en la evaluación formativa y sumativa al organizar tareas, rúbricas y retroalimentación en una plataforma web y móvil; al mismo tiempo, debe considerarse que su adopción puede generar efectos secundarios como dependencia tecnológica o menor interacción tradicional si no se equilibra bien con la práctica pedagógica habitual.

TABLA 6 IMPACTOS POSITIVOS VS NEGATIVOS

Impactos positivos	Impactos negativos
Mejor organización del trabajo docente: permite centralizar tareas, calificaciones, rúbricas y entregas en un solo lugar.	Mayor carga emocional o estrés para docentes que no acepten cambios rápidos.
Mayor eficiencia en evaluación: reduce tiempo invertido en calificar, registrar y compartir resultados.	Posible dependencia excesiva de la herramienta, disminuyendo el juicio pedagógico tradicional.
Fácil adopción para profesores de 40 a 60 años: una interfaz minimalista e intuitiva puede disminuir la barrera tecnológica.	Disminución de interacción cara a cara si se usa demasiado la plataforma para comunicación y evaluación.
Mejora de la retroalimentación formativa: facilita dar comentarios claros y estructurados a los estudiantes.	Falta de soporte en momentos críticos, lo que puede afectar el flujo de trabajo en clases o evaluaciones.
Acceso multiplataforma: disponibilidad web y móvil ayuda a docentes y alumnos a usar la herramienta desde distintos dispositivos.	Problemas de confianza si los datos de calificaciones o evaluaciones no se muestran con claridad.
Transparencia en el seguimiento: los estudiantes y docentes pueden ver el progreso académico de forma más ordenada.	

En general, el proyecto puede aumentar la productividad docente y el seguimiento del aprendizaje, pero también puede traer riesgos de resistencia al cambio, reducción del juicio educativo

tradicional y necesidad de soporte constante, por lo que su éxito dependerá de un diseño realmente intuitivo, capacitación adecuada y un uso que complemente, no reemplace, la enseñanza presencial.

Factibilidad

La siguiente sección presenta la factibilidad del proyecto MaxGrade, mostrando los costos estimados para mantener la plataforma y el equipo necesario para el desarrollo. Se divide en factibilidad económica, donde se consideran servicios y APIs, y factibilidad financiera, donde se detallan los equipos de trabajo.

Económica:

En la parte económica se identifican los gastos recurrentes del proyecto: hosting de base de datos, servidor, API de detección de IA y servicio de transcripción de audio a texto. Estos recursos suman alrededor de \$1,466.46 MXN mensuales, lo que permite dimensionar el presupuesto operativo necesario para mantener la aplicación activa.

TABLA 7 FACTIBILIDAD ECONÓMICA COSTOS DEL SERVIDOR

Nombre	Descripción	Cantidad	Costo/mes
Supabase	Host de base de datos	1	\$434.45 MXN
Vercel	Host del servidor	1	\$347.56 MXN
Sapling AI	API para detección de IA en documentos de Microsoft 365	1	\$434.45 MXN
Groq	API para transcripción de audio a texto	1	\$250 MXN
		Total	\$1466.46 MX

TABLA 8 FACTIBILIDAD ECONÓMICA COSTO PERSONAL

Nombre	Rol Equipo	Gastos /mes
Luis Emiliano Hernández Carmona	Programador Movil	\$8,000 MXN

Correa Estrada Arantxa	Analista	\$9,000 MXN
Velazquez López Romario	Diseñador	\$8,500 MXN
Ruiz Vázquez Máximo	Programador Front-Web	\$9,500MXN
Salinas Ramos Luis Miguel	Programador web	\$9,500 MXN
	Total	\$35,5000

Financiera:

TABLA 9 FACTIBILIDAD FINANCIERA

Nombre	Descripción	Cantidad	Costo
MSI Bravo 17C7V	Laptop	1	\$44,000 MXN
Inspiron 15 3535	Laptop	1	\$11,529 MXN
Computadora	De escritorio	3	\$24,000 MXN
	Total		\$79,529 MXN

Rentabilidad

Para determinar la viabilidad financiera del proyecto, se ha realizado una proyección de los costos operativos mensuales frente al modelo de ingresos previsto mediante el plan de suscripción Premium.

1.Estructura de Costos

El proyecto contempla dos categorías principales de egresos recurrentes mensuales:

Costos Operativos (Infraestructura y Servicios):

Incluyen el soporte de bases de datos (Supabase), alojamiento del servidor (Vercel), y el consumo de APIs especializadas (Sapling AI para detección de IA y Groq para transcripción). Estos servicios suman un total de \$1,466.46 MXN mensuales.

Gastos de Personal: Corresponden a la retribución del equipo de desarrollo, análisis y diseño, con un costo mensual total de \$35,500.00 MXN.

Sumando ambos, el costo mensual de operación del proyecto asciende a \$36,966.46 MXN.

2. Modelo de Ingresos y Punto de Equilibrio

La estrategia de monetización se basa en un esquema de suscripción Premium con un costo de \$119.00 MXN.

Para alcanzar el punto de equilibrio mensual y cubrir la totalidad de los gastos operativos y de personal, el proyecto requiere la captación y mantenimiento de al menos 311 usuarios activos bajo el plan Premium.

3. Consideraciones de Rentabilidad

Este análisis demuestra que el proyecto presenta una estructura de costos clara y manejable. La rentabilidad dependerá directamente de la escalabilidad del modelo de negocio; una vez superados los 311 usuarios necesarios para cubrir los costos fijos, cada suscripción adicional representará un margen de utilidad directa para la sostenibilidad y crecimiento del proyecto.

Backlog

ID	Nombre	Descripción (Enunciado)
HU-001	Navegación: Visualizar página de inicio	Como visitante, quiero navegar y visualizar la página de inicio para entender el propósito de MaxGrade.
HU-002	Cuenta: Crear cuenta de usuario	Como visitante, quiero crear una cuenta para poder acceder a los módulos del sistema.
HU-003	Sesión: Iniciar sesión	Como usuario, quiero iniciar sesión con mis credenciales para acceder a mi perfil y funciones personalizadas.
HU-004	Perfil: Modificar perfil	Como usuario, quiero modificar los datos permitidos de mi perfil para personalizar mi experiencia.
HU-005	Cuenta: Eliminar cuenta	Como usuario, quiero eliminar mi cuenta para darme de baja del servicio y remover mis datos del sistema.
HU-028	Clases: Crear clase	Como profesor, quiero crear una clase nueva asignándole nombre, sección y materia para empezar a organizar mi curso.
HU-008	Clases: Inscribirse a clase	Como alumno, quiero ingresar un código alfanumérico para inscribirme instantáneamente en una clase.
HU-035	Tablón: Publicar anuncio	Como profesor, quiero publicar anuncios generales para toda la clase en el flujo principal.
HU-036	Tareas: Crear tarea	Como profesor, quiero crear una tarea con título, descripción, puntaje máximo, fecha de entrega y archivos adjuntos.
HU-018	Tareas: Entregar tarea (cargar archivos)	Como alumno, quiero cargar archivos (PDF, DOCX, JPG) desde mi dispositivo para entregar trabajos.
HU-045	Tareas: Calificar entrega	Como profesor, quiero ingresar una calificación numérica y retroalimentación a la entrega de un alumno y devolverla.
HU-006	Sesión: Cerrar sesión	Como usuario, quiero cerrar sesión de forma segura para proteger mi información en dispositivos compartidos.

HU-007	Perfil: Visualizar datos del alumno (Tu trabajo)	Como alumno, quiero acceder a mi sección 'Tu trabajo' para revisar tareas entregadas, calificaciones y comentarios del profesor.
HU-009	Clases: Listar clases activas	Como alumno, quiero ver una lista de todas mis clases activas con la materia y el profesor.
HU-011	Tablón: Ver publicaciones	Como alumno, quiero ver las publicaciones del profesor en el tablón para estar al tanto de avisos.
HU-012	Tablón: Comentar anuncio	Como alumno, quiero publicar un comentario en un anuncio público para resolver dudas que mis compañeros también puedan tener.
HU-017	Tareas: Ver detalles de tarea	Como alumno, quiero ver instrucciones, fecha de entrega y puntaje máximo de una tarea para entender qué debo hacer.
HU-020	Tareas: Anular entrega	Como alumno, quiero poder anular una entrega ya enviada (antes de la fecha límite) para realizar correcciones de último minuto.
HU-024	Tareas: Ver nota	Como alumno, quiero ver mi nota numérica una vez que el profesor devuelva la tarea, mostrando la escala (ej. 80/100).
HU-029	Clases: Configurar banner	Como profesor, quiero elegir un tema o subir una imagen de banner para que mi clase sea fácilmente identificable.
HU-030	Clases: Generar código de invitación	Como profesor, quiero que se genere automáticamente el código de invitación al crear la clase.
HU-037	Tareas: Asignar puntaje máximo	Como profesor, quiero asignar un valor numérico máximo a una tarea para establecer la escala de calificación.
HU-038	Tareas: Definir fecha de entrega	Como profesor, quiero establecer una fecha y hora de entrega para que el sistema marque automáticamente envíos tardíos.
HU-042	Tareas: Adjuntar archivos a tarea	Como profesor, quiero adjuntar archivos locales, enlaces o videos de YouTube a mis tareas para enriquecer el contenido.
HU-043	Tareas: Ver entregas de alumnos	Como profesor, quiero ver una lista de quiénes han entregado la tarea y quiénes la tienen pendiente.
HU-044	Tareas: Previsualizar archivos entregados	Como profesor, quiero abrir y previsualizar los archivos enviados por los alumnos dentro de la plataforma.
HU-046	Tareas: Enviar retroalimentación	Como profesor, quiero enviar comentarios de retroalimentación directos al alumno sobre su trabajo.
HU-047	Tareas: Devolver trabajo calificado	Como profesor, quiero 'Devolver' el trabajo para que el alumno reciba su nota y pueda ver mis comentarios.
HU-041	Materiales: Crear temas/unidades	Como profesor, quiero crear 'Temas' o 'Unidades' para agrupar tareas y materiales de forma lógica dentro de la clase.
HU-022	Calendario: Visualizar fechas de entrega	Como alumno, quiero visualizar mis fechas de entrega en un calendario para organizar mi tiempo semanal.

HU-034	Clases: Dar de baja alumno	Como profesor, quiero dar de baja a un alumno de la clase para mantener la lista actualizada.
HU-049	Evaluación: Ver cuadrícula de notas	Como profesor, quiero ver una cuadrícula con todos los alumnos y todas sus notas para tener una visión general del rendimiento.
HU-014	Mensajes: Privados en tarea (desfase desde Sprint 1)	Como alumno, quiero enviar y recibir mensajes privados con mi profesor dentro de una tarea específica para recibir retroalimentación personalizada.
HU-016	Materiales: Filtrar contenido por tema	Como alumno, quiero filtrar el contenido por 'Temas' o 'Unidades' para encontrar rápidamente la información de un examen.
HU-019	Tareas: Marcar entregada sin archivo	Como alumno, quiero marcar una tarea como 'Entregada' aunque no tenga archivos adjuntos, para avisar al profesor que terminé.
HU-021	Tareas: Lista consolidada de todas las materias	Como alumno, quiero ver una lista consolidada de todas mis tareas (Asignadas, Sin entregar, Completadas) de todas las materias.
HU-023	Tareas: Historial de entregas y estados	Como alumno, quiero ver un resumen histórico de mis entregas y el estado de cada una (Calificado, Entregado, Tarde).
HU-025	Notificaciones: Entrega calificada	Como alumno, quiero recibir una notificación una vez que el profesor devuelva y califique mi tarea.
HU-048	Evaluación: Crear rúbrica de calificación	Como profesor, quiero crear criterios de evaluación detallados para calificar de forma más objetiva y transparente.
HU-050	Evaluación: Historial individual de alumno	Como profesor, quiero hacer clic en un alumno específico para ver su historial completo de entregas y faltas en mi materia.
HU-032	Clases: Copiar clase existente	Como profesor, quiero copiar una clase existente (con sus materiales y tareas) para reutilizar la estructura en un nuevo ciclo escolar.
HU-039	Tareas: Programar publicación	Como profesor, quiero programar la fecha y hora de publicación de una tarea para que aparezca automáticamente en el futuro.
HU-010	Clases: Organizar tarjetas (drag & drop)	Como alumno, quiero arrastrar y soltar mis tarjetas de clase para organizarlas según mi horario diario, con persistencia entre sesiones.
HU-027	Materiales: Subir recursos de estudio	Como profesor, quiero subir recursos de lectura o videos que no requieren entrega para que los alumnos estudien.
HU-033	Clases: Archivar clase finalizada	Como profesor, quiero archivar clases finalizadas para limpiar mi panel principal sin perder la información histórica.
HU-040	Tareas: Guardar borrador de tarea	Como profesor, quiero guardar una tarea a medio terminar como borrador para continuar editándola después.
HU-051	Evaluación: Detección de uso de IA	Como profesor, quiero comprobar si un alumno realizó su tarea con IA para poder anularla si procede.

HU-052	Evaluación: Comentario por voz a texto	Como profesor, quiero usar un transcriptor de voz a texto para dictar comentarios de retroalimentación sin escribirlos.
---------------	--	---

Artefactos

Nombre	Descripción	Enlace	Tipo
Requerimientos Funcionales	Listado de lo que el sistema debe hacer		Análisis
Requerimientos No Funcionales	Criterios de calidad y restricciones del proyecto		Análisis
Especificación de Requerimiento	Requisitos con su contexto, condiciones de aceptación, flujo principal, flujos alternos y postcondiciones		Análisis
Plan de Riesgos	Analisis de las amenazas al proyecto, con su impacto, y las acciones preventivas para reducir fallos o problemas.		Planeación

Roles

En el organigrama del proyecto MaxGrade se muestra la estructura de trabajo para la aplicación escolar diseñada para docentes de 40 a 60 años, indicando cómo se distribuyen las responsabilidades entre los integrantes del equipo.

ILUSTRACIÓN 3 ORGANIGRAMA

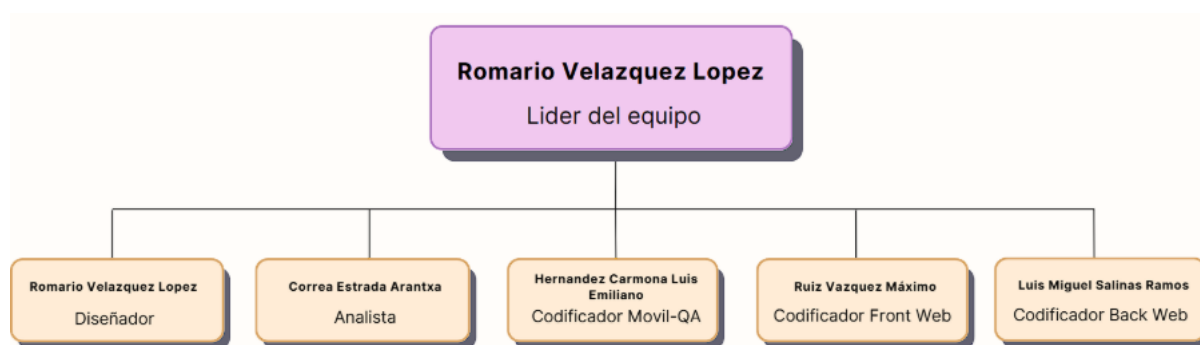


Diagrama de Gantt:

El proyecto se planificó en dos fases principales que abarcan desde el 12 de marzo hasta el 12 de junio de 2026.

La primera fase (Fase de Desarrollo) detalla la ejecución de tres sprints cronológicos independientes, orientados a la construcción del sistema y la redacción de los primeros apartados teóricos. La segunda fase (Fase de Cierre y Validación) contempla tres etapas críticas adicionales: Recolección de datos y liberación de capítulos, un periodo dedicado a Correcciones de software (bugs en plataformas web y móvil) junto con la integración documental, y finalmente, el bloque de Conclusiones, destinado a la redacción del Capítulo 4 y la sustentación final del proyecto.

ILUSTRACIÓN 4 DIAGRAMA DE GANTT 1

					12 al 21 de Marzo del 2026							12 al 27 de Abril del 2026							27 de Abril al 9 de Mayo del 2026																
Tarea	Asignado/a	Progreso	Inicio	Fin	12	13	16	17	18	19	20	21	12	13	14	15	16	17	20	21	22	23	24	27	27	28	29	30	1	4	5	6	7	8	9
Sprint 1		100%	12/03/2026	21/03/2026																															
Corregir Capitulo 1 de la Tesina	Arantxa	100%	12/03/2026	20/03/2026																															
Codificar Back	Luis Miguel	100%	12/03/2026	20/03/2026																															
Diseñar Front	Romario	100%	12/03/2026	20/03/2026																															
Codificar Front	Luis Miguel	100%	12/03/2026	20/03/2026																															
Sprint 2		100%	12/04/2026	27/04/2026																															
Finalizar Capitulo 2 de la Tesina	Todos	100%	12/04/2026	27/04/2026																															
Diseño de componentes de interfaz	Romario	100%	12/04/2026	20/04/2026																															
Codificar Back	Luis Miguel	100%	12/04/2026	27/04/2026																															
Crear Front	Maximo	100%	12/04/2026	23/04/2026																															
Adaptar Base de Datos	Luis Miguel	100%	12/04/2026	23/04/2026																															
Sprint 3		100%	27/04/2026	09/05/2026																															
Desarrollar Front de Soporte	Maximo	100%	27/04/2026	04/05/2026																															
Codificar Back de Soporte	Luis Miguel	100%	27/04/2026	04/05/2026																															
Realizar Pruebas	Luis Emiliano	100%	27/04/2026	09/05/2026																															
Finalizar Figma	Romario	100%	27/04/2026	30/04/2026																															

ILUSTRACIÓN 5 DIAGRAMA DE GANTT 2

					18 al 27 de Mayo							28 de Mayo al 12 de Junio del 2026												
Tarea	Asignado/a	Progreso	Inicio	Fin	18	19	20	21	22	25	26	27	28	29	1	2	3	4	5	8	9	10	11	12
Recolección de datos		0%	18/05/2026	30/05/2026																				
Recolección de datos	Arantxa	0%	18/05/2026	30/05/2026																				
Liberar Capitulo 1	Luis Miguel	0%	18/05/2026	21/05/2026																				
Liberar Capitulo 2	Romario	0%	22/05/2026	27/05/2026																				
Liberar Capitulo 3	Luis Miguel	0%	28/05/2026	30/05/2026																				
Corecciones		0%	01/06/2026	05/06/2026																				
Arreglar bugs Web	Luis Miguel	0%	01/06/2026	05/06/2026																				
Arreglar bugs Movil	Luis Emiliand	0%	01/06/2026	05/06/2026																				
Integrar Capítulos Tesina	Todos	0%	01/06/2026	05/06/2026																				
Conclusiones		100%	08/06/2026	12/06/2026																				
Redacción Capitulo 4	Todos	0%	08/06/2026	11/06/2026																				
Presentación Formal del Proyecto	Todos	0%	12/06/2026	12/06/2026																				

Ruta Crítica:

La ruta crítica para nuestro proyecto contempla una duración total de 98 días (llegando a la presentación formal en el día 96), iniciando con el Plan Estratégico (A) y vinculando de manera consecutiva las fases de Estudio del Mercado (B), Ruta Crítica (F), Especificación de Requerimientos (H), Diagrama de Actividades (J), Manual de Usuario (L), Figma (N), Codificación de Proyecto (O) y Pruebas (P), hasta concluir con la Presentación formal de Su Proyecto (R).

Debido a que estas actividades clave carecen de holgura, cualquier desviación en su ejecución impactará directamente en la fecha de entrega final programada para el cierre del proyecto.

Actividades:

ILUSTRACIÓN 6 ACTIVIDADES RUTA CRÍTICA

- A Plan Estratégico

B Estudio del Mercado

C Documentación de Requerimientos

D Administración de Riesgos

E Diagrama de Gantt

F Ruta Critica

G Diagrama de Casos de Uso

H Especificación de Requerimientos

I Diagrama de Clases
- J Diagrama de Actividades

K Diagrama Entidad - Relación

L Manual de Usuario

M Manual de Instalación

N Figma

Ñ Mapa de Navegación

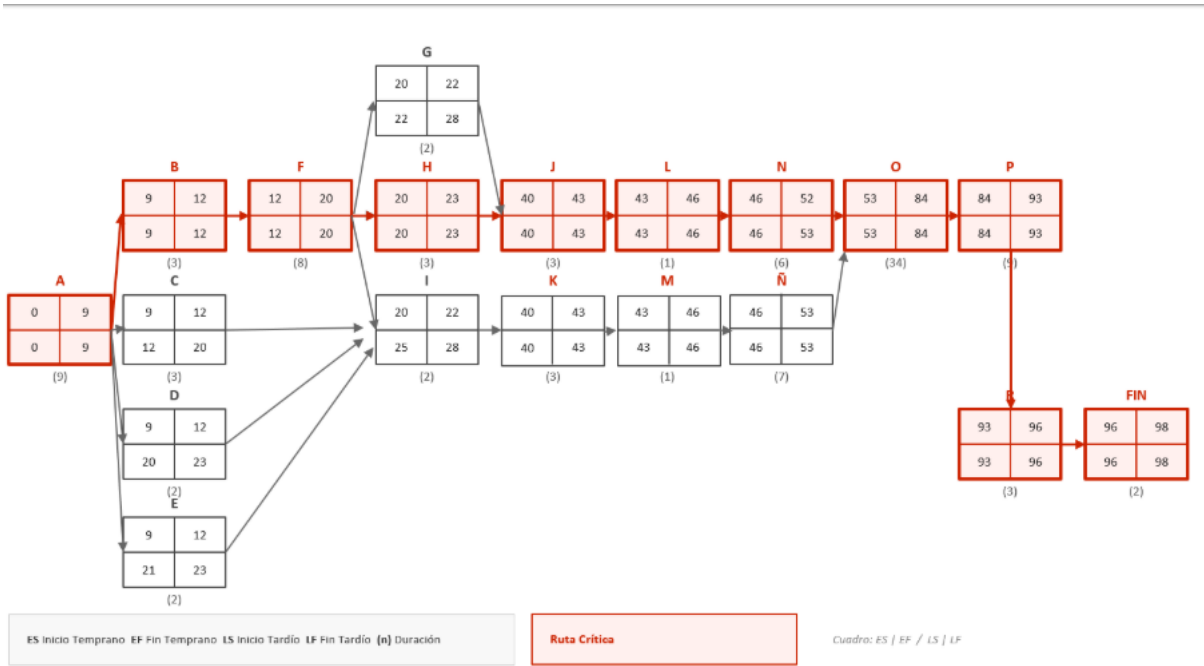
O Codificación de Proyecto

P Pruebas

R Presentación formal de Su Proyecto

Ruta:

ILUSTRACIÓN 7 RUTA CRÍTICA



Metodología SCRUM

Durante el desarrollo de la aplicación MaxGrade, se utilizó una metodología ágil denominada SCRUM, ya que ofrecía una mayor flexibilidad, entregas rápidas y continuas, ciclos iterativos (Sprints) y una colaboración continua. Se explica con más detalle en los siguientes puntos:

- Flexibilidad:** Esta metodología permitió que el desarrollo del proyecto se adaptara a cambios inesperados en los requerimientos o en la arquitectura del sistema. Al no ser un plan rígido de inicio a fin.

- **Entregas rápidas y continuas:** En lugar de esperar a tener la aplicación finalizada para realizar pruebas, se optó por un modelo de entrega continua. Esto permitió tener versiones funcionales de módulos específicos de forma temprana.
- **Sprints:** El desarrollo se dividió en bloques de tiempo fijos o "Sprints" (generalmente de 2 semanas). Al inicio de cada uno, se seleccionaron tareas para cada integrante específicas del backlog.
- **Colaboración Continua:** Con reuniones periódicas (como las diarias), se comunicó sobre el estado del proyecto. Esta interacción constante facilitó el repartir de las tareas pendientes y futuras sobre el proyecto.

Planeaciones de Sprint

Esta sección está destinada a dar un informe sobre lo que se realizó durante cada sprint, en un tiempo determinado para las entregas continuas. También se realizaron reuniones diarias denominadas "Dailys" con la herramienta de Microsoft Teams con el fin de mantener una comunicación constante. A continuación, una tabla que indica los rangos de las fechas para cada sprint.

TABLA 10 FECHAS SPRINT

Sprint	Rango de Fechas
S1	Jueves 12 de marzo a viernes 20 de marzo
S2	Lunes 13 de abril a lunes 27 de abril
S3	Lunes 27 de abril a sábado 9 de mayo

Los casos de prueba fueron ejecutados sobre la plataforma web y están organizados por módulo. Para ello, se planearon utilizar los siguientes ambientes:

TABLA 11 AMBIENTES DE PRUEBA

Integrante	Dispositivo de Pruebas
Correa Estrada Arantxa (AMWB01)	16 GB RAM · 1 TB · Windows 11
Velazquez Lopez Romario (AMWB02)	16GB RAM · 1TB · Windows 11
Salinas Ramos Luis Miguel (AMWB03)	PC · 8 GB RAM · 512 GB · Windows 10
Ruiz Vázquez Máximo (AMWB04)	PC · 16GB RAM · 1.4TB · Windows 10

Hernández Carmona Luis Emiliano (AMWB05)	Laptop · 16 GB RAM·954 GB·Windows 11
--	--------------------------------------

Reportes de Sprint

Sprint 1

Objetivo: Finalizar la implementación de las Historias de Usuario correspondientes al Sprint 1, asegurando la consistencia entre el desarrollo técnico y los prototipos de Figma. Asimismo, la correcta visualización de los endpoints, generar la documentación de pruebas funcionales del bloque inicial e identificar las actividades pendientes para el siguiente sprint.

Historias de Usuario Finalizadas – 28

TABLA 12 HISTORIAS DE USUARIO SPRINT 1

ID	Nombre	Descripción (Enunciado)
HU-001	Navegación: Visualizar página de inicio	Como visitante, quiero navegar y visualizar la página de inicio para entender el propósito de MaxGrade.
HU-002	Cuenta: Crear cuenta de usuario	Como visitante, quiero crear una cuenta para poder acceder a los módulos del sistema.
HU-003	Sesión: Iniciar sesión	Como usuario, quiero iniciar sesión con mis credenciales para acceder a mi perfil y funciones personalizadas.
HU-004	Perfil: Modificar perfil	Como usuario, quiero modificar los datos permitidos de mi perfil para personalizar mi experiencia.
HU-005	Cuenta: Eliminar cuenta	Como usuario, quiero eliminar mi cuenta para darme de baja del servicio y remover mis datos del sistema.
HU-028	Clases: Crear clase	Como profesor, quiero crear una clase nueva asignándole nombre, sección y

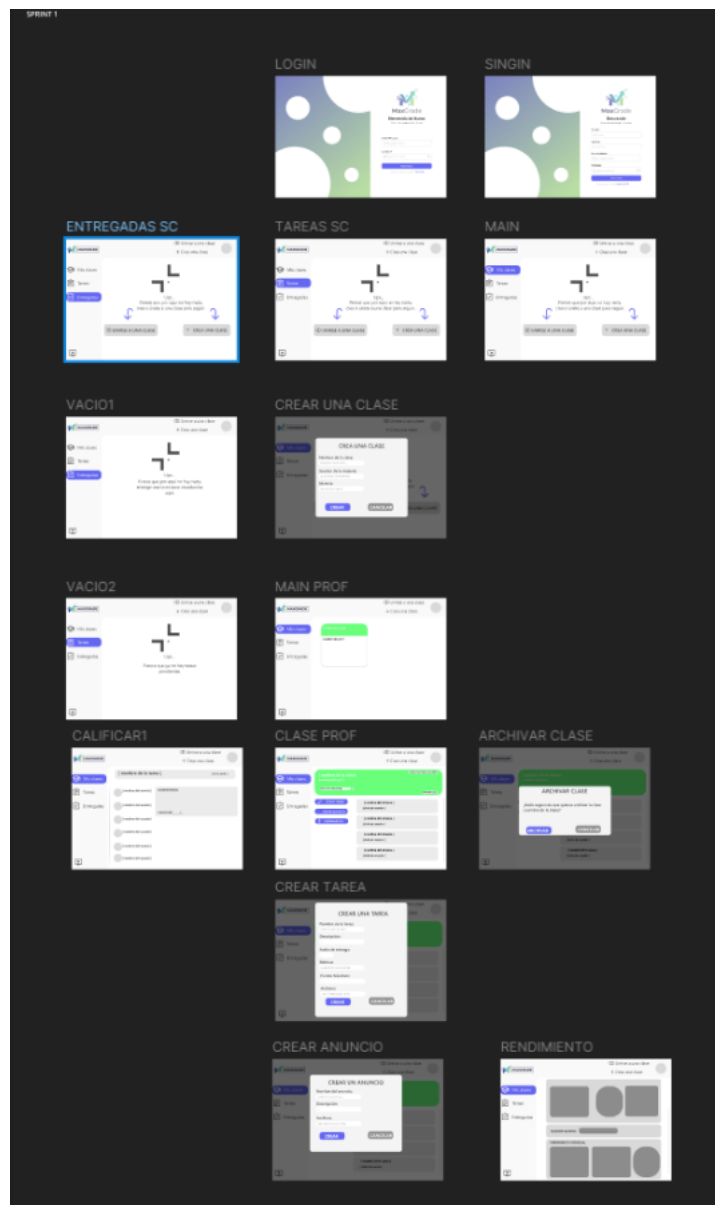
		materia para empezar a organizar mi curso.
HU-008	Clases: Inscribirse a clase	Como alumno, quiero ingresar un código alfanumérico para inscribirme instantáneamente en una clase.
HU-035	Tablón: Publicar anuncio	Como profesor, quiero publicar anuncios generales para toda la clase en el flujo principal.
HU-036	Tareas: Crear tarea	Como profesor, quiero crear una tarea con título, descripción, puntaje máximo, fecha de entrega y archivos adjuntos.
HU-018	Tareas: Entregar tarea (cargar archivos)	Como alumno, quiero cargar archivos (PDF, DOCX, JPG) desde mi dispositivo para entregar trabajos.
HU-045	Tareas: Calificar entrega	Como profesor, quiero ingresar una calificación numérica y retroalimentación a la entrega de un alumno y devolverla.
HU-006	Sesión: Cerrar sesión	Como usuario, quiero cerrar sesión de forma segura para proteger mi información en dispositivos compartidos.
HU-007	Perfil: Visualizar datos del alumno (Tu trabajo)	Como alumno, quiero acceder a mi sección 'Tu trabajo' para revisar tareas entregadas, calificaciones y comentarios del profesor.
HU-009	Clases: Listar clases activas	Como alumno, quiero ver una lista de todas mis clases activas con la materia y el profesor.
HU-011	Tablón: Ver publicaciones	Como alumno, quiero ver las publicaciones del profesor en el tablón para estar al tanto de avisos.
HU-012	Tablón: Comentar anuncio	Como alumno, quiero publicar un comentario en un anuncio público para

		resolver dudas que mis compañeros también puedan tener.
HU-017	Tareas: Ver detalles de tarea	Como alumno, quiero ver instrucciones, fecha de entrega y puntaje máximo de una tarea para entender qué debo hacer.
HU-020	Tareas: Anular entrega	Como alumno, quiero poder anular una entrega ya enviada (antes de la fecha límite) para realizar correcciones de último minuto.
HU-024	Tareas: Ver nota	Como alumno, quiero ver mi nota numérica una vez que el profesor devuelva la tarea, mostrando la escala (ej. 80/100).
HU-029	Clases: Configurar banner	Como profesor, quiero elegir un tema o subir una imagen de banner para que mi clase sea fácilmente identificable.
HU-030	Clases: Generar código de invitación	Como profesor, quiero que se genere automáticamente el código de invitación al crear la clase.
HU-037	Tareas: Asignar puntaje máximo	Como profesor, quiero asignar un valor numérico máximo a una tarea para establecer la escala de calificación.
HU-038	Tareas: Definir fecha de entrega	Como profesor, quiero establecer una fecha y hora de entrega para que el sistema marque automáticamente envíos tardíos.
HU-042	Tareas: Adjuntar archivos a tarea	Como profesor, quiero adjuntar archivos locales, enlaces o videos de YouTube a mis tareas para enriquecer el contenido.
HU-043	Tareas: Ver entregas de alumnos	Como profesor, quiero ver una lista de quiénes han entregado la tarea y quiénes la tienen pendiente.

HU-044	Tareas: Previsualizar archivos entregados	Como profesor, quiero abrir y previsualizar los archivos enviados por los alumnos dentro de la plataforma.
HU-046	Tareas: Enviar retroalimentación	Como profesor, quiero enviar comentarios de retroalimentación directos al alumno sobre su trabajo.
HU-047	Tareas: Devolver trabajo calificado	Como profesor, quiero 'Devolver' el trabajo para que el alumno reciba su nota y pueda ver mis comentarios.

- Diseño:

ILUSTRACIÓN 8 DISEÑO SPRINT 1



Durante el primer sprint se desarrollaron las pantallas correspondientes a las historias de usuario de los docentes, poniendo de forma visual todas las funciones y opciones a las que los docentes tendrán acceso dentro de la aplicación. Esto permitió definir con claridad la estructura de la plataforma y la forma en que los usuarios interactuarán con el sistema.

Hubo algunas pantallas que se desfazaron hasta el segundo Sprint y es por ello por lo que no se incluyeron en la imagen. Enlace al [figma](#)

- Documentación de Código — Endpoints Principales (11 HUs)

TABLA 13 DOCUMENTACIÓN CÓDIGO SPRINT 1

HU-001 Navegación: Visualizar página de inicio		
Método / Ruta	Descripción	Estatus
GET /	Renderiza la página principal estática (index.html) para visitantes no autenticados.	Entrega final Sprint 1
HU-002 Cuenta: Crear cuenta de usuario		
Método / Ruta	Descripción	Estatus
GET /registro	Renderiza el formulario de registro.	Entrega final Sprint 1
POST /api/auth/register	Crea la cuenta validando campos obligatorios y unicidad de email; retorna datos del usuario.	Entrega final Sprint 1
HU-003 Sesión: Iniciar sesión		
Método / Ruta	Descripción	Estatus
GET /login	Renderiza el formulario de inicio de sesión.	Entrega final Sprint 1
POST /api/auth/login	Autentica al usuario, genera JWT (24 h) y establece cookie de sesión segura; redirige al Dashboard.	Entrega final Sprint 1
HU-004 Perfil: Modificar perfil		
Método / Ruta	Descripción	Estatus

GET /perfil	Renderiza la página de perfil con los datos actuales del usuario.	Entrega final Sprint 1
PUT /api/auth/update-profile	Actualiza nombre, apellido y email del usuario autenticado.	Entrega final Sprint 1
POST /api/auth/update-avataar	Sube o reemplaza la foto de perfil en el bucket 'avataares' de Supabase Storage.	Entrega final Sprint 1
DELETE /api/auth/delete-avataar	Elimina la foto de perfil de la base de datos y del storage.	Entrega final Sprint 1
HU-005 Cuenta: Eliminar cuenta		
Método / Ruta	Descripción	Estatus
DELETE /api/auth/delete-account	Elimina permanentemente la cuenta y todos sus datos en cascada (entregas, tareas, anuncios, inscripciones). Acción irreversible.	Entrega final Sprint 1
HU-028 Clases: Crear clase		
Método / Ruta	Descripción	Estatus
POST /api/classes/create	Crea la clase, registra al profesor autenticado como propietario y genera automáticamente el código de acceso hexadecimal de 6 caracteres.	Entrega final Sprint 1
HU-008 Clases: Inscribirse a clase		
Método / Ruta	Descripción	Estatus

POST /api/classes/join	Valida el código de acceso y registra al alumno autenticado como estudiante en la clase correspondiente; redirige al tablón.	Entrega final Sprint 1
HU-035 Tablón: Publicar anuncio		
Método / Ruta	Descripción	Estatus
POST /api/posts	Crea un nuevo anuncio en la clase especificada; el autor se asigna del usuario autenticado.	Entrega final Sprint 1
DELETE /api/posts/:id	Elimina un anuncio; solo el autor o el profesor propietario pueden ejecutarlo.	Entrega final Sprint 1
HU-036 Tareas: Crear tarea		
Método / Ruta	Descripción	Estatus
POST /api/assignments	Crea la tarea en la clase indicada. Soporta 'puntos_maximos' (default 100), 'fecha_entrega' obligatoria y archivo guía opcional en Supabase Storage.	Entrega final Sprint 1
GET /api/assignments/class/:claseId	Lista todas las tareas de una clase con sus metadatos.	Entrega final Sprint 1
DELETE /api/assignments/:id	Elimina una tarea; solo el profesor propietario puede ejecutarlo.	Entrega final Sprint 1
HU-018 Tareas: Entregar tarea (cargar archivos)		
Método / Ruta	Descripción	Estatus

POST /api/assignments/submit	Recibe la entrega del alumno con archivo adjunto opcional; registra fecha/hora de envío y marca como tardía si supera la fecha límite.	Entrega final Sprint 1
HU-045 Tareas: Calificar entrega		
Método / Ruta	Descripción	Estatus
PUT /api/assignments/grade/:id	Guarda la calificación numérica y el comentario del profesor; actualiza el estado de la entrega a 'Calificada' y lo hace visible para el alumno.	Entrega final Sprint 1

- **Pruebas:**

Para cada Sprint se listan todas las Historias de Usuario completadas junto con su documentación de pruebas funcionales.

Los siguientes 10 escenarios cubren los flujos de seguridad (registro, login, JWT), creación de clase con su código de invitación, publicación de anuncios, creación de tareas, entrega por parte del alumno y calificación por el profesor, correspondientes a las 28 Historias de Usuario finalizadas en este Sprint.

TABLA 14 PRUEBAS SPRINT 1

ID	HUs Relacionadas	Descripción del Escenario	Pasos de Ejecución	Resultado Esperado	Estatus
CP-E2E-S1-01	HU-002 / HU-003	Registro de nuevo usuario e inicio de sesión exitoso con generación y uso de JWT.	1. POST /api/auth/register (nombre válido, correo único, contraseña ≥ 7 chars) → 201 Created + datos de usuario. 2. POST /api/auth/login con las mismas credenciales → 200 OK + JWT en cookie HttpOnly. 3. GET /dashboard con JWT válido → 200 OK, vista del dashboard cargada correctamente.	API retorna 201 en registro y 200 en login. El JWT es generado con expiración 24 h, almacenado en cookie HttpOnly. El dashboard carga sin redirección.	
CP-E2E-	HU-003	Intento de login con credenciales	1. POST /api/auth/login con contraseña incorrecta → 401. 2.	API retorna 400 o 401 en todos los	

S1-02		incorrectas, campos vacíos e inyección SQL/XSS.	POST /api/auth/login con todos los campos vacíos → 400. 3. POST /api/auth/login con correo: ' OR '1'='1 → 400. 4. POST /api/auth/login con correo: <script>alert(1)</script> → 400.	casos negativos. Los payloads maliciosos son sanitizados. La interfaz muestra mensaje de error sin ejecutar código. La base de datos no es afectada.	
CP-E2E-S1-03	HU-002	Registro con datos inválidos: correo duplicado, formato de email incorrecto y contraseña débil.	1. POST /api/auth/register con correo ya registrado → 409 Conflict. 2. POST /api/auth/register con correo: alrome@gmail.co (dominio inválido) → 400. 3. POST /api/auth/register con contraseña: '1' (muy corta) → 400 + mensaje 'Contraseña débil'. 4. POST /api/auth/register con nombre vacío → 400.	API retorna el código HTTP semánticamente correcto (400 o 409) con mensaje descriptivo. No se crea ningún registro en base de datos. La UI muestra la alerta junto al campo incorrecto.	
CP-E2E-S1-04	HU-028 / HU-030	Profesor crea clase con datos válidos y el sistema genera automáticamente el código de invitación.	1. POST /api/classes/create (nombre: 'Cálculo Diferencial', sección: 'Sexto', materia: 'Matemáticas') → 201 Created. 2. Verificar en respuesta que código hex de 6 caracteres fue generado. 3. GET /dashboard → nueva clase visible en la lista del profesor. 4. GET /clase/:id → código de invitación mostrado en la vista de la clase.	La clase aparece en el dashboard del profesor. El código de invitación es alfanumérico único, de 6 caracteres. La UI muestra el código de forma destacada para compartir con alumnos.	
CP-E2E-S1-05	HU-028	Validación de campos obligatorios y límites de caracteres al crear clase.	1. POST /api/classes/create sin nombre → 400 'Nombre obligatorio'. 2. POST /api/classes/create con nombre de 50 caracteres (excede límite 25) → 400. 3. POST /api/classes/create con sección de 50 caracteres →	Los campos obligatorios (nombre, sección) disparan 400 si faltan o exceden el límite. La materia es	

			400. 4. POST /api/classes/create con materia vacía (campo opcional) → 201 OK.	opcional: si se omite, la clase se crea correctamente. Mensajes de validación visibles en UI.	
CP-E2E-S1-06	HU-008	Alumno se inscribe a clase con código válido, con código inválido y con código de clase inexistente.	1. POST /api/classes/join con código válido (MAT101) → 200 OK, alumno inscrito. 2. GET /clase/:id → clase visible en dashboard del alumno. 3. POST /api/classes/join con código inexistente (XYZ999) → 404. 4. POST /api/classes/join con código vacío → 400. 5. POST /api/classes/join con mismo código (ya inscrito) → 409.	Con código válido el alumno queda inscrito y ve la clase en su dashboard. Los códigos inválidos retornan el error correspondiente sin inscripción. Doble inscripción bloqueada con 409.	
CP-E2E-S1-07	HU-035	Profesor publica anuncio en el tablón de la clase. Alumno visualiza el anuncio y comenta.	1. POST /api/posts (clase: MAT101, contenido: 'Hola alumnos, examen el viernes') → 201. 2. Alumno: GET /clase/:id → anuncio visible en el tablón. 3. Alumno: POST /api/posts/:id/comments con comentario → 201. 4. POST /api/posts con contenido vacío → 400. 5. DELETE /api/posts/:id (profesor) → 200, anuncio eliminado.	El anuncio aparece en tiempo real en el tablón. El alumno puede comentar. El contenido vacío es rechazado. Solo el autor o el profesor propietario pueden eliminar publicaciones.	
CP-E2E-S1-08	HU-036 / HU-038 / HU-042	Profesor crea tarea con título, fecha de entrega futura, puntaje máximo y archivo adjunto (PDF).	1. POST /api/assignments (título: 'Ejercicios U1', instrucciones, puntos: 100, fecha: +7 días, archivo PDF) → 201. 2. GET /api/assignments/class/:id → tarea listada con todos sus metadatos. 3. Alumno: GET /clase/:id/tareas → tarea visible con fecha y puntaje. 4. POST /api/assignments con fecha pasada → 400. 5. POST	La tarea creada aparece para los alumnos con su fecha de entrega, instrucciones y archivo descargable. Fecha pasada y puntaje fuera de rango (1–100)	

			/api/assignments con puntos_maximos: 1000 → 400.	son rechazados con 400 y mensaje descriptivo.	
CP-E2E-S1-09	HU-018 / HU-020	Alumno entrega tarea con archivo adjunto y posteriormente anula la entrega antes del plazo.	1. POST /api/assignments/submit (tarea: 'Ejercicios U1', archivo DOCX) → 201, estado 'Entregada'. 2. GET /api/assignments/:id → timestamp de envío registrado, estado confirmado. 3. POST /api/assignments/submit/:id/cancel (antes de fecha límite) → 200, estado vuelve a 'Pendiente'. 4. Alumno vuelve a entregar → 201, nuevo timestamp registrado.	La entrega se registra con timestamp. La anulación es posible antes del límite y deja la tarea en estado pendiente. La re-entrega funciona correctamente. La UI actualiza el estado visualmente.	
CP-E2E-S1-10	HU-045 / HU-046 / HU-047	Profesor califica la entrega de un alumno, escribe retroalimentación y la devuelve al alumno.	1. PUT /api/assignments/grade/:entrega_id (calificación: 85, comentario: 'Buen trabajo, revisar ejercicio 3') → 200. 2. Estado de entrega cambia a 'Calificada'. 3. Alumno: GET /clase/:id/tu-trabajo → nota 85/100 y comentarios visibles. 4. Verificar que la nota se refleja correctamente en la escala (85/100).	La calificación y retroalimentación son visibles para el alumno una vez devuelta. El estado cambia a 'Calificada'. El profesor puede editar la nota antes de devolver. La escala numérica es consistente.	

Durante la ejecución de pruebas del Sprint 1 se identificaron los siguientes defectos. Todos fueron corregidos antes del cierre del Sprint:

TABLA 15 ERRORES CORREGIDOS SPRINT 1

ID	Módulo / HU	Descripción del Defecto	Corrección Aplicada	Estatus
DEF-S1-01	Registro (HU-002)	El sistema permitía registrar correos con formato inválido	Se añadió validación de regex en el middleware de Express	

		(ej. alrome@gmail.co , sin dominio completo).	y en el frontend antes del envío.	
DEF-S1-02	Clases (HU-028)	El sistema no validaba el límite de 25 caracteres en nombre y sección de la clase, creando registros con desbordamiento.	Se implementó validación de longitud máxima en el controlador de creación de clases (backend + frontend).	
DEF-S1-03	Sesión (HU-003)	El sistema permitía iniciar sesión con emojis en el campo nombre, generando tokens con datos corruptos.	Se añadió sanitización de caracteres no alfanuméricos Unicode en el controlador de autenticación.	
DEF-S1-04	Sesión (HU-003)	Contraseñas con más de 90 caracteres o que contenían emojis permitían el inicio de sesión sin error.	Se estableció límite máximo de 18 caracteres (compatible con bcrypt) y bloqueo de caracteres no ASCII.	

Durante este primer sprint se llevó a cabo el trabajo más pesado del proyecto, completando 34 tareas planificadas. En este ciclo, se priorizó el desarrollo de la infraestructura base del sistema, las funcionalidades esenciales y la lógica de la aplicación. Sin embargo, esto generó que tareas como elementos específicos de diseño y componentes de confirmación fueran postergadas para el sprint 2, debido a falta de comunicación en el equipo, mala organización y la gran carga de trabajo que teníamos.

Sprint 2

Objetivo: Finalizar el desarrollo de las funciones de organización por temas, el sistema de calendario y el panel de rendimiento académico, asegurando la correcta ejecución de los endpoints de gestión de alumnos y evaluación. En el ámbito documental, concluir la redacción del Capítulo 2 de la tesina e iniciar la descripción metodológica del Capítulo 3, integrando los diagramas y diseños técnicos desarrollados hasta el momento.

Historias de Usuario Finalizadas – 12

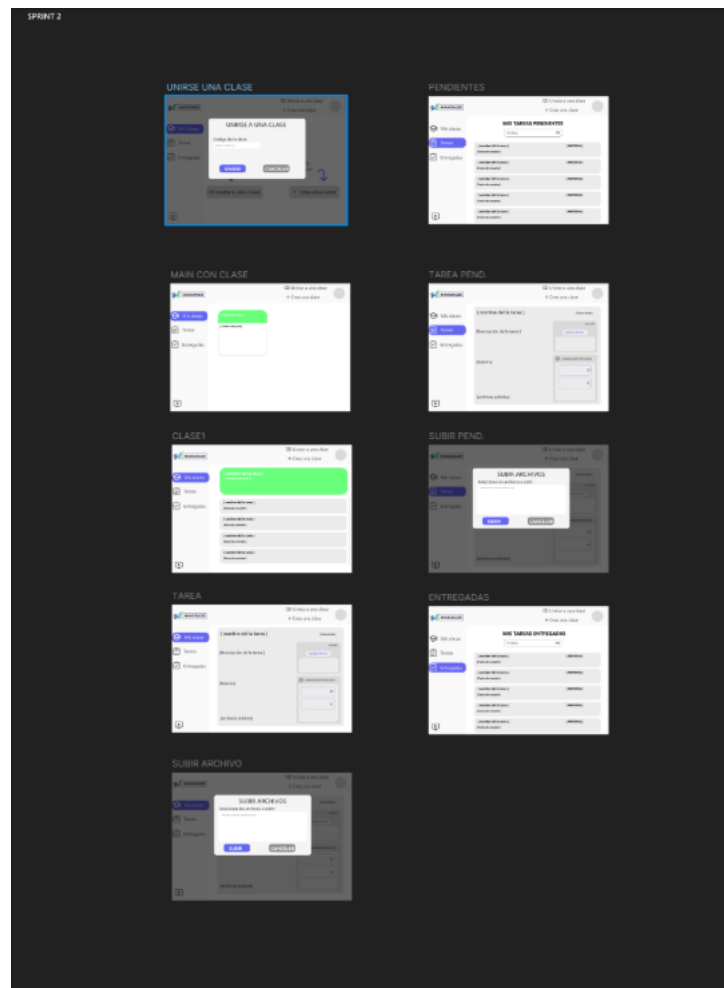
TABLA 16 HISTORIAS DE USUARIO SPRINT 2

ID	Nombre	Descripción (Enunciado)
HU-041	Materiales: Crear temas/unidades	Como profesor, quiero crear 'Temas' o 'Unidades' para agrupar tareas y materiales de forma lógica dentro de la clase.
HU-022	Calendario: Visualizar fechas de entrega	Como alumno, quiero visualizar mis fechas de entrega en un calendario para organizar mi tiempo semanal.
HU-034	Clases: Dar de baja alumno	Como profesor, quiero dar de baja a un alumno de la clase para mantener la lista actualizada.
HU-049	Evaluación: Ver cuadrícula de notas	Como profesor, quiero ver una cuadrícula con todos los alumnos y todas sus notas para tener una visión general del rendimiento.
HU-014	Mensajes: Privados en tarea (desfase desde Sprint 1)	Como alumno, quiero enviar y recibir mensajes privados con mi profesor dentro de una tarea específica para recibir retroalimentación personalizada.
HU-016	Materiales: Filtrar contenido por tema	Como alumno, quiero filtrar el contenido por 'Temas' o 'Unidades' para encontrar rápidamente la información de un examen.

HU-019	Tareas: Marcar entregada sin archivo	Como alumno, quiero marcar una tarea como 'Entregada' aunque no tenga archivos adjuntos, para avisar al profesor que terminé.
HU-021	Tareas: Lista consolidada de todas las materias	Como alumno, quiero ver una lista consolidada de todas mis tareas (Asignadas, Sin entregar, Completadas) de todas las materias.
HU-023	Tareas: Historial de entregas y estados	Como alumno, quiero ver un resumen histórico de mis entregas y el estado de cada una (Calificado, Entregado, Tarde).
HU-025	Notificaciones: Entrega calificada	Como alumno, quiero recibir una notificación una vez que el profesor devuelva y califique mi tarea.
HU-048	Evaluación: Crear rúbrica de calificación	Como profesor, quiero crear criterios de evaluación detallados para calificar de forma más objetiva y transparente.
HU-050	Evaluación: Historial individual de alumno	Como profesor, quiero hacer clic en un alumno específico para ver su historial completo de entregas y faltas en mi materia.

- Diseño:

ILUSTRACIÓN 9 DISEÑO SPRINT 2



Durante el segundo sprint se desarrollaron las pantallas correspondientes a las historias de usuario de los alumnos, incluyendo de forma visual las funciones y opciones a las que ellos tendrán acceso dentro de la aplicación. Asimismo, en este periodo se terminaron de realizar las pantallas que se encontraban desfasadas. Cabe mencionar que algunas de las pantallas de este segundo sprint se tuvieron que incluir en la imagen del primer sprint, debido a que pertenecían al apartado de las historias de usuario de los docentes.

- Documentación de Código - Endpoints Principales (4 HUs)

TABLA 17 DOCUMENTACIÓN CÓDIGO SPRINT 2

HU-041 Materiales: Crear temas/unidades		
Método / Ruta	Descripción	Estatus
POST /api/assignments	El campo 'unit_id' al crear una tarea la agrupa bajo el tema correspondiente.	Entrega final Sprint 2
GET /api/assignments/class/:classId/unit/:unitId	Obtiene todas las tareas de una unidad específica dentro de una clase, ordenadas por fecha de entrega.	Entrega final Sprint 2
HU-022 Calendario: Visualizar fechas de entrega		
Método / Ruta	Descripción	Estatus
GET /calendario	Renderiza el calendario con tareas pendientes y vencidas de todas las clases del usuario. Soporta parámetros 'mes' y 'año' para navegar entre meses.	Entrega final Sprint 2
HU-034 Clases: Dar de baja alumno		
Método / Ruta	Descripción	Estatus
DELETE /api/classes/:classId/students/:studentId	Elimina la inscripción del alumno; solo el profesor propietario puede ejecutarlo. El alumno pierde acceso inmediato.	Entrega final Sprint 2

GET /clase/:id/personas	Renderiza la página con la lista de miembros de la clase separados en profesores y alumnos.	Entrega final Sprint 2
HU-049 Evaluación: Ver cuadrícula de notas		
Método / Ruta	Descripción	Estatus
GET /clase/:id/rendimiento	Renderiza el panel de rendimiento con estadísticas generales, lista de estudiantes y mapa JSON de desempeño individual. Resalta celdas sin entrega.	Entrega final Sprint 2

- Pruebas:

Los siguientes 6 escenarios cubren los flujos incorporados en el Sprint 2: creación y filtrado de unidades, calendario de entregas, baja de alumnos, cuadrícula de rendimiento, mensajería privada y gestión del estado de entregas, correspondientes a las 12 Historias de Usuario finalizadas en este Sprint.

TABLA 18 PRUEBAS SPRINT 2

ID	HUs Relacionadas	Descripción del Escenario	Pasos de Ejecución	Resultado Esperado	Estatus
CP-E2E-S2-01	HU-041	Profesor crea unidades/temas para organizar el contenido de la clase. Alumno filtra tareas por unidad.	1. POST /api/units (clase: MAT101, número: 1, nombre: 'Termoquímica', descripción opcional) → 201. 2. POST /api/units duplicado (número 1 en misma clase) → 400 'Unidad ya existe'. 3. POST /api/units sin nombre → 400 'Nombre obligatorio'. 4. POST /api/assignments con unit_id asignado → tarea agrupada bajo la unidad. 5. Alumno: GET	Las unidades se crean y agrupan tareas lógicamente. No se permiten números de unidad duplicados en la misma clase. El filtrado por	✓

			/api/assignments/class/:id/unit/1 → solo tareas de esa unidad.	unidad retorna únicamente las tareas correspondientes .	
CP-E2E-S2-02	HU-022	Alumno visualiza sus fechas de entrega en el calendario mensual y navega entre meses.	1. GET /calendario → vista del mes actual con tareas marcadas en sus fechas de entrega. 2. GET /calendario?mes=6&año=2026 → navegación a junio. 3. Verificar que tareas de múltiples clases aparecen consolidadas. 4. Verificar distinción visual entre tareas pendientes (azul) y vencidas (rojo).	El calendario muestra correctamente las fechas de entrega de todas las clases del alumno. La navegación por mes funciona sin errores. El código de colores diferencia el estado de cada tarea.	
CP-E2E-S2-03	HU-034	Profesor da de baja a un alumno de la clase. El alumno pierde acceso inmediato.	1. GET /clase/:id/personas → lista de alumnos inscritos visible. 2. DELETE /api/classes/:classId/students/:studentId (profesor autenticado) → 200 OK. 3. Alumno dado de baja: GET /clase/:id → 403 Forbidden, redirigido a dashboard. 4. DELETE por usuario que no es el profesor propietario → 403. 5. Verificar que las entregas del alumno permanecen en el historial del profesor.	El alumno es eliminado de la lista de inscritos y pierde acceso inmediato. Sus entregas históricas permanecen. Solo el profesor propietario puede ejecutar la baja. Otros usuarios reciben 403.	
CP-E2E-S2-04	HU-049	Profesor visualiza la cuadrícula de notas con todos los alumnos y el estado de cada tarea.	1. GET /clase/:id/rendimiento → cuadrícula con filas por alumno y columnas por tarea. 2. Verificar que celdas sin entrega se resaltan visualmente (color diferente). 3. Verificar que entregas tardías tienen indicador visual. 4. Verificar que el promedio de cada alumno se calcula correctamente sobre tareas calificadas.	La cuadrícula muestra el estado de cada alumno por tarea: Calificada, Entregada, Tardía, Sin entregar. Los promedios son correctos. Las	

				celdas vacías están resaltadas. El panel carga en tiempo razonable.	
CP-E2E-S2-05	HU-014	Alumno envía mensaje privado al profesor dentro de una tarea. El profesor responde desde el panel de calificación.	1. POST /api/assignments/:id/messages (alumno) con texto de duda → 201. 2. Profesor: GET /api/assignments/:id/messages → mensaje visible en panel de calificación. 3. Profesor: POST /api/assignments/:id/messages con respuesta → 201. 4. Alumno: GET tarea → respuesta del profesor visible en el hilo privado. 5. Alumno de otra clase intenta leer el mensaje → 403.	Los mensajes privados son visibles solo para el alumno y el profesor de esa tarea. El hilo se mantiene ordenado cronológicamente. Acceso externo bloqueado con 403.	
CP-E2E-S2-06	HU-019 / HU-021 / HU-023	Alumno marca tarea como entregada sin archivo, consulta lista consolidada de todas sus tareas y revisa historial de entregas.	1. POST /api/assignments/submit (sin archivo adjunto, solo marca) → 201, estado 'Entregada'. 2. GET /tareas (vista consolidada) → lista todas las tareas de todas las clases con sus estados. 3. Verificar secciones: Asignadas, Sin entregar, Completadas. 4. GET /historial → entregas con estados: Calificado, Entregado, Tarde.	La entrega sin archivo es válida y actualiza el estado. La vista consolidada agrupa correctamente tareas de múltiples clases. El historial refleja todos los estados con sus timestamps.	

Durante la ejecución de pruebas del Sprint 2 se identificaron los siguientes defectos. Todos fueron corregidos]antes del cierre del Sprint:

TABLA 19 ERRORES CORREGIDOS SPRINT 2

ID	Módulo / HU	Descripción del Defecto	Corrección Aplicada	Estatus
DEF-S2-01	Unidades (HU-041)	El sistema permitía crear unidades con número	Se añadió restricción de unicidad (clase_id + numero_unidad) en la base	

		duplicado dentro de la misma clase sin mostrar error.	de datos y validación en el controlador.	
DEF-S2-02	Personas (HU-034)	Un alumno podía acceder a /clase/:id/personas de clases en las que no estaba inscrito.	Se añadió middleware de verificación de membresía antes de renderizar la página de personas.	

Durante este sprint logramos completar lo postergado a diseño de Sprint 1 y al mismo tiempo agregamos funciones nuevas a la aplicación a través de 27 tareas. Aunque fue el sprint con mayor peso por lo heredado del anterior, se lograron completar todas las tareas por la buena comunicación y organización del equipo.

Sprint 3

Objetivo: Concluir el desarrollo de las aplicaciones web y móvil mediante la implementación de herramientas de gestión avanzada, como el copiado de clases, la programación de tareas y la organización de interfaz mediante drag & drop. Asimismo, finalizar la redacción del Capítulo 3 de la tesina, documentando historias de usuario, diseño realizado en figma, documentación de código y pruebas realizadas.

Historias de usuario Finalizadas – 9

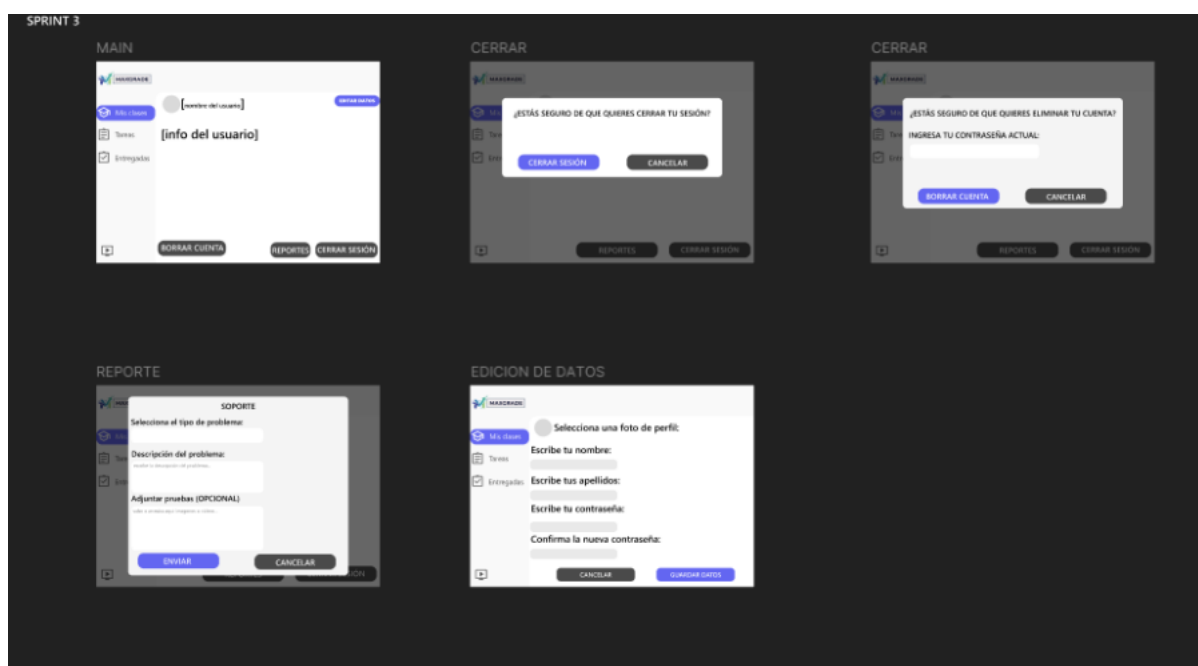
TABLA 20 HISTORIAS DE USUARIO SPRINT 3

ID	Nombre	Descripción (Enunciado)
HU-032	Clases: Copiar clase existente	Como profesor, quiero copiar una clase existente (con sus materiales y tareas) para reutilizar la estructura en un nuevo ciclo escolar.
HU-039	Tareas: Programar publicación	Como profesor, quiero programar la fecha y hora de publicación de una tarea para que aparezca automáticamente en el futuro.
HU-010	Clases: Organizar tarjetas (drag & drop)	Como alumno, quiero arrastrar y soltar mis tarjetas de clase para organizarlas

		según mi horario diario, con persistencia entre sesiones.
HU-031	Clases: Restablecer código de invitación	Como profesor, quiero restablecer el código de invitación para controlar el acceso de nuevos alumnos, invalidando el anterior.
HU-027	Materiales: Subir recursos de estudio	Como profesor, quiero subir recursos de lectura o videos que no requieren entrega para que los alumnos estudien.
HU-033	Clases: Archivar clase finalizada	Como profesor, quiero archivar clases finalizadas para limpiar mi panel principal sin perder la información histórica.
HU-040	Tareas: Guardar borrador de tarea	Como profesor, quiero guardar una tarea a medio terminar como borrador para continuar editándola después.
HU-051	Evaluación: Detección de uso de IA	Como profesor, quiero comprobar si un alumno realizó su tarea con IA para poder anularla si procede.
HU-052	Evaluación: Comentario por voz a texto	Como profesor, quiero usar un transcriptor de voz a texto para dictar comentarios de retroalimentación sin escribirlos.

- Diseño:

ILUSTRACIÓN 10 DISEÑO SPRINT 3



Durante el tercer sprint, el trabajo se enfocó en el desarrollo de las pantallas relacionadas con la configuración de usuario. Cabe mencionar que las historias de usuario correspondientes a los docentes y alumnos ya habían sido incluidas como una actualización del diseño en las imágenes de los sprints anteriores.

- Documentación de Código:

TABLA 21 DOCUMENTACIÓN CÓDIGO SPRINT 3

HU-032 Clases: Copiar clase existente		
Método / Ruta	Descripción	Estatus
POST /api/classes/create	Se crea una nueva clase y se replican en cascada los elementos de la original: temas, tareas en estado borrador y materiales. No incluye lista de alumnos ni publicaciones.	Entrega final Sprint 3
HU-039 Tareas: Programar publicación		
Método / Ruta	Descripción	Estatus

POST /api/assignments	El campo 'fecha_publicacion' permite programar la tarea; el backend filtra las no publicadas para los alumnos hasta que llegue el momento configurado.	Entrega final Sprint 3
------------------------------	--	------------------------

- Pruebas:

Los siguientes 5 escenarios cubren los flujos avanzados de gestión de clases y tareas incorporados en el Sprint 3: copiado de clase en cascada, publicación programada, restablecimiento de código, archivado y borradores, correspondientes a las 9 Historias de Usuario finalizadas en este Sprint.

TABLA 22 PRUEBAS SPRINT 3

ID	HUs Relacionadas	Descripción del Escenario	Pasos de Ejecución	Resultado Esperado	Estatus
CP-E2E-S3-01	HU-032	Profesor copia una clase existente con sus unidades y tareas para un nuevo ciclo escolar.	1. Profesor selecciona 'Copiar clase' en la clase 'Cálculo Diferencial'. 2. POST /api/classes/create (con flag de copia y source_class_id) → 201, nueva clase creada. 3. GET /clase/:id_nueva → verificar que las unidades de la original están presentes. 4. Verificar que las tareas copiadas tienen estado 'Borrador'. 5. Verificar que la lista de alumnos de la nueva clase está vacía. 6. Verificar que las publicaciones/anuncios NO se copiaron.	La clase copiada hereda la estructura de unidades y tareas (en borrador). Sin alumnos ni anuncios. El código de invitación es nuevo y único. El profesor puede editar las tareas copiadas antes de publicarlas.	
CP-E2E-S3-02	HU-039	Profesor programa la publicación de una tarea para una fecha/hora futura. Validación de invisibilidad	1. POST /api/assignments con fecha_publicacion = ahora +2 horas → 201, estado 'Programada'. 2. Alumno: GET /clase/:id/tareas antes de la hora → tarea NO aparece en la lista. 3. Simular avance	El backend filtra tareas con fecha_publicacion futura para los alumnos. Al superar la hora programada, la tarea aparece automáticamente sin	

		antes de la hora programada.	de tiempo (ajustar fecha_publicacion a pasado en BD de prueba). 4. Alumno: GET /clase/:id/tareas → tarea ahora visible. 5. Alumno puede ver detalles y hacer entrega normalmente.	intervención del profesor. La entrega funciona con normalidad.	
CP-E2E-S3-03	HU-031	Profesor restablece el código de invitación de la clase. El código anterior queda invalidado.	1. Anotar código original de la clase (ej. 'A1B2C3'). 2. POST /api/classes/:id/reset-code (profesor autenticado) → 200, nuevo código generado. 3. POST /api/classes/join con código anterior 'A1B2C3' → 404 'Código inválido'. 4. POST /api/classes/join con nuevo código → 200, inscripción exitosa. 5. Alumno no-propietario intenta reiniciar → 403.	El código anterior queda invalidado inmediatamente. El nuevo código es único. Solo el profesor propietario puede ejecutar el restablecimiento. Los alumnos que ya estaban inscritos no son afectados.	
CP-E2E-S3-04	HU-033	Profesor archiva una clase finalizada. La clase desaparece del dashboard principal pero conserva su historial.	1. PUT /api/classes/:id/archive (profesor) → 200, estado cambia a 'Archivada'. 2. GET /dashboard → clase archivada NO aparece en la lista principal. 3. GET /dashboard?tab=archivadas → clase visible en sección de archivadas. 4. Alumnos: GET /dashboard → clase no visible. 5. GET /clase/:id/rendimiento en clase archivada → historial de notas accesible (solo lectura).	La clase archivada desaparece del dashboard activo pero es accesible desde la sección de archivadas. Los datos históricos (notas, entregas) son consultables en modo solo lectura.	
CP-E2E-S3-05	HU-040	Profesor guarda una tarea como borrador, la edita y posteriormente	1. POST /api/assignments con estado 'borrador' → 201, tarea no visible para alumnos. 2. PUT	Los borradores no son visibles para los alumnos. El profesor puede editarlos	

		la publica para los alumnos.	/api/assignments/:id (actualizar título e instrucciones) → 200. 3. GET /clase/:id/tareas (alumno) → borrador NO aparece. 4. PUT /api/assignments/:id/publish → 200, estado cambia a 'Publicada'. 5. GET /clase/:id/tareas (alumno) → tarea ahora visible.	libremente. Al publicar, la tarea aparece en la vista del alumno con sus datos actualizados.	
--	--	------------------------------	---	--	--

Durante la ejecución de pruebas del Sprint 3 se identificaron los siguientes defectos. Todos fueron corregidos y re-verificados antes del cierre del Sprint:

TABLA 23 ERRORES CORREGIDOS SPRINT 3

ID	Módulo / HU	Descripción del Defecto	Corrección Aplicada	Estatus
DEF-S3-01	Copiar clase (HU-032)	Al copiar una clase, los archivos adjuntos de las tareas no se duplicaban en Supabase Storage, generando enlaces rotos.	Se implementó copia de archivos en el bucket 'tareas' de Supabase Storage durante el proceso de copiado en cascada.	
DEF-S3-02	Programar publicación (HU-039)	El filtro de fecha_publicacion no excluía correctamente las tareas programadas para alumnos en el endpoint de listado.	Se agregó condición WHERE fecha_publicacion <= NOW() OR fecha_publicacion IS NULL en la consulta de tareas para alumnos.	

La fase de pruebas de software de MaxGrade se ejecutó de manera sistemática en tres Sprints, abarcando pruebas funcionales de interfaz y escenarios de seguridad. Dando así las siguientes conclusiones:

- Seguridad validada en Sprint 1: todos los intentos de inyección SQL y XSS fueron bloqueados. La protección JWT funciona correctamente en todas las rutas privadas del sistema.
- Validaciones robustas en Sprint 1 y 2: campos obligatorios, límites de caracteres, formatos de email, rangos de puntaje y permisos por rol retornan mensajes descriptivos al usuario.

- Control de acceso en Sprint 2 y 3: el código 403 Forbidden se implementó y validó correctamente para todas las operaciones de administración (dar de baja, archivar, resetear código) que requieren ser propietario de la clase.
- Funcionalidades avanzadas en Sprint 3: el copiado de clase en cascada (incluyendo archivos en Storage) y la publicación programada funcionan sin exponer borradores o contenido no publicado a los alumnos.
- Defectos gestionados: los 8 defectos identificados (4 en S1, 2 en S2, 2 en S3) fueron corregidos y re-verificados dentro del mismo Sprint, sin arrastrar deuda técnica a sprints posteriores.

Este fue el sprint más corto en duración y por lo tanto de tareas (solo 9), con el enfoque a mejorar los últimos detalles de lo ya hecho en los 2 sprints anteriores. Se dedicó principalmente a pruebas de los módulos de Alumno, Clases y Tareas, se corrigieron bugs en el flujo de entrega y calendario, se finalizó el Figma y el Capítulo 2 y 3 de la tesina, finalmente se desarrolló el módulo de soporte (back y front). Todo cerró en Finalizada, sin deuda pendiente, utilizamos este sprint final para estabilizarnos antes de comenzar con la recolección de datos para el capítulo 4 de la Tesina.

Ceremonias

Esta sección está destinada a dar un informe sobre las ceremonias realizadas en cada sprint, otorgando la retrospectiva general de lo que el equipo noto en sus deficiencias y áreas de mejora. Para documentar las ceremonias se utilizó la herramienta de Figma con el fin de llevar un orden.

Ceremonia – Sprint 1

Período: jueves 12 de marzo – viernes 20 de marzo de 2025

Duración: 8 días

En cuanto a lo que el equipo debía dejar de hacer, se identificó una tendencia común a dispersarse entre múltiples tareas sin concluir las, así como a postergar actividades para los últimos días del sprint. También se señaló la falta de revisión conjunta de documentos antes de entregarlos, lo que generaba errores que podían haberse evitado con una revisión en equipo.

Entre los aspectos positivos, destacó la conclusión de los documentos de legado del sistema y el análisis FODA, los cuales fueron completados de manera satisfactoria. Además, el equipo logró identificar correctamente las historias de usuario pendientes, lo que sentó una base sólida para los siguientes sprints.

Como áreas de mejora para futuros sprints, el equipo acordó mejorar la organización individual y colectiva, apoyarse mutuamente ante dudas en lugar de cargar la responsabilidad en un solo integrante, y organizar reuniones más frecuentes para revisar el avance general del proyecto.

En lo individual, los principales retos giraron en torno a la revisión y actualización del capítulo 1 de la tesina, que no pudo concluirse en tiempo por falta de organización y coordinación. Algunos integrantes también señalaron dificultades para cumplir con sus entregas debido a distracciones externas o falta de conocimientos técnicos.

Como aprendizajes del sprint, el equipo destacó la importancia de revisar los documentos más de una vez antes de entregarlos, mantener el Jira actualizado conforme se avanza, y adoptar herramientas de inteligencia artificial para facilitar las tareas de programación. Asimismo, se reconoció el valor de reunirse de forma constante para revisar el estado general del proyecto.

El Jugador Más Valioso del sprint fue Luis Miguel, reconocido por la mayoría del equipo por avanzar de forma consistente en distintas áreas sin correcciones, y por su contribución en la aplicación web y en la revisión de documentos.

Ceremonia – Sprint 2

Período: lunes 13 de abril – lunes 27 de abril de 2025

Duración: 2 semanas

Respecto a lo que se debía dejar de hacer, el equipo coincidió en la necesidad de eliminar la procrastinación, tanto a nivel individual como colectivo. Se identificó también la tendencia a distraerse durante las sesiones de trabajo en clase y a no leer los documentos con detenimiento antes de entregarlos, lo que seguía generando errores evitables.

Entre los logros del sprint, destacaron la organización de las tareas y el buen funcionamiento de las dailys como mecanismo de seguimiento. El equipo resaltó que la forma de distribuir las tareas pendientes fue más eficiente que en el sprint anterior, y que la comunicación mejoró mucho, reflejando un mayor enfoque colectivo en las correcciones de la tesina.

Para el siguiente sprint, el equipo propuso no retrasar actividades, mejorar la estimación de tiempos, y fomentar la acción preguntar abiertamente las dudas en lugar de quedarse con ellas. También se habló de la necesidad de que todos los integrantes se preparen para poder asumir distintas tareas, especialmente las de programación.

A nivel individual, los principales problemas del sprint fueron el retraso en el diseño de la aplicación web, que tuvo que rehacerse para adaptarse a los objetivos del proyecto, y la postergación de las pruebas por dejar la programación para el final. Algunos integrantes también señalaron dificultades para corregir el capítulo 1 de la tesina por falta de conocimientos o de organización con el equipo.

Los aprendizajes del sprint se enfocaron en la importancia de planificar las actividades individuales con anticipación, pedir ayuda a los compañeros en lugar de intentar resolver todo de manera independiente, y comunicarse de forma más asertiva sin generar conflictos. También se destacó la necesidad de mantener respaldos del código fuente.

El Jugador Más Valioso del sprint fue nuevamente Luis Miguel, reconocido por sus avances en la documentación y en el desarrollo de la aplicación web. Romario también fue mencionado por varios integrantes por su papel clave en identificar y visibilizar los problemas del equipo para poder corregirlos.

Ceremonia – Sprint 3

Período: lunes 27 de abril – sábado 9 de mayo de 2025

Duración: 2 semanas

En esta retrospectiva final, el equipo identificó como principal hábito a eliminar el trabajo individualizado sin revisión colectiva, así como la tendencia a subestimar los tiempos disponibles para cada actividad. También se señaló la procrastinación como un patrón que persistió durante todo el proyecto y que afectó la calidad y puntualidad de algunas entregas.

Los logros más relevantes del sprint fueron la conclusión de la aplicación web, la corrección de los capítulos 1 y 2 de la tesina y el avance al capítulo 3, así como la finalización de la codificación del sistema. El equipo consideró que este sprint fue el más productivo en términos de entregables concretos.

Como propuestas de mejora para proyectos futuros, el equipo enfatizó la necesidad de revisar al menos dos veces antes de su presentación, establecer estimaciones de tiempo más precisas y asegurarse de que todos los integrantes puedan asumir distintas responsabilidades técnicas, particularmente en programación.

A nivel individual, los retos más mencionados fueron la dificultad para mantenerse al día con los entregables de la tesina, la falta de involucramiento pleno en las actividades del equipo por parte de algunos integrantes, y errores derivados de modificar módulos del sistema sin comprenderlos completamente.

Entre los aprendizajes finales del proyecto, el equipo destacó la importancia de distribuir las tareas según las capacidades de cada integrante, establecer plazos de revisión más estrictos, y continuar explorando el uso de herramientas de inteligencia artificial para optimizar el trabajo de programación. También se resaltó el aprendizaje técnico en áreas como JWT para la autorización de recursos.

El Jugador Más Valioso del sprint fue Arantxa, reconocida por su contribución en la documentación y por su papel en la organización del equipo durante el cierre del proyecto. Luis Miguel también fue destacado por completar el diseño de la aplicación web e integrar funciones adicionales, además de colaborar activamente en la documentación de la tesina.

Capítulo 4: Resultados

El presente capítulo documenta los resultados obtenidos durante la fase de pruebas de usuario de MaxGrade, así como el análisis de mejora continua realizado a partir de la retroalimentación recopilada. Para ello, se aplicó el Ciclo de Deming (Planificar–Hacer–Verificar–Actuar) como metodología iterativa orientada a la calidad del producto, permitiendo al equipo identificar áreas de oportunidad, validar los objetivos planteados y establecer una hoja de ruta para el desarrollo futuro del sistema.

Prueba de Usuario

Tipo de Prueba: Alfa

Para el proyecto MaxGrade se llevó a cabo una prueba de usuario de tipo Alfa, es decir, una evaluación realizada en un ambiente controlado con un grupo cerrado de participantes previamente seleccionados. Este tipo de prueba se caracteriza porque el acceso al sistema está restringido a un conjunto definido de usuarios, las sesiones se realizan en un entorno supervisado donde el equipo de desarrollo puede observar y registrar el comportamiento de los participantes, y el objetivo principal es identificar errores funcionales, problemas de usabilidad y áreas de mejora antes de una liberación más amplia del producto.

Se determinó realizar una prueba de tipo Alfa —y no Beta— debido a que el sistema se encontraba en una fase de madurez funcional suficiente para ser evaluado, pero aún requería validación controlada antes de exponerse a un entorno sin supervisión. Dado que el público objetivo es docente de un rango de edad específico (40–60 años) con posibles limitaciones en el manejo de herramientas digitales, el ambiente controlado permitió al equipo intervenir oportunamente ante confusiones y recopilar retroalimentación más precisa.

Características de la Prueba

La prueba de usuario Alfa de MaxGrade se realizó bajo las siguientes condiciones:

Ambiente: Laboratorio de cómputo del CECyT 9 "Juan de Dios Bátiz", con acceso controlado a la plataforma desplegada en Vercel.

Total, de participantes: 80 personas, distribuidas de la siguiente manera:

- 26 docentes activos del plantel, en un rango de edad de 30 a 60 años.
- 54 alumnos representantes de grupo, seleccionados por el coordinador académico.

Tareas asignadas: Cada participante recibió un guion de tareas estructurado según su rol en el sistema (docente o alumno). Los docentes debieron crear una clase, publicar una tarea con archivos adjuntos, calificar una entrega y generar retroalimentación por voz. Los alumnos debieron inscribirse a una clase mediante código, entregar una tarea y revisar su calificación. Todos los participantes navegaron por el módulo de perfil y configuración de cuenta.

Instrumento de recopilación: Al finalizar cada sesión, los participantes completaron un cuestionario de satisfacción con escala Likert del 1 al 5, más un apartado de comentarios abiertos.

Ciclo de Deming Aplicado a MaxGrade

El Ciclo de Deming, también conocido como ciclo PHVA (Planificar, Hacer, Verificar, Actuar), es una metodología iterativa de mejora continua ampliamente utilizada en la gestión de calidad de productos y servicios. Su aplicación en el contexto de MaxGrade permitió al equipo estructurar el proceso de recopilación y análisis de retroalimentación de manera sistemática, asegurando que los hallazgos de las pruebas de usuario se tradujeran en acciones concretas de mejora.

Planificar

Objetivo: Obtener la mayor cantidad de retroalimentación cualitativa y cuantitativa por parte de los docentes y alumnos del CECyT 9 respecto a la usabilidad, funcionalidad e intuitividad de MaxGrade, con el fin de identificar fricciones en los flujos de uso y priorizar mejoras para iteraciones futuras.

Para alcanzar este objetivo, el equipo planificó las siguientes estrategias de recopilación de información:

Registro de logs del sistema: Se habilitó un módulo de registro de actividad en el backend de MaxGrade que almacenó, para cada sesión de prueba, los endpoints consultados, los tiempos de respuesta por módulo, los errores HTTP generados (400, 403, 404, 409, 500) y las acciones completadas versus abandonadas por cada usuario. Este log permitió identificar patrones de comportamiento sin depender exclusivamente de la memoria o percepción de los participantes.

Módulo de soporte: Previo a las sesiones de prueba, se activó el módulo de soporte técnico de MaxGrade, que permite a los usuarios enviar reportes de incidencias directamente desde la plataforma. Durante las sesiones Alfa, se registraron 23 tickets de soporte, cuyas categorías y resoluciones se detallan en el apartado de Hacer. Este módulo proporcionó evidencia directa de los puntos de fricción experimentados por los usuarios en tiempo real.

Cuestionario post-sesión: Se diseñó un instrumento de evaluación con 15 ítems organizados en tres dimensiones: usabilidad general (claridad de la interfaz, facilidad de navegación, comprensión de iconos), funcionalidad percibida (completitud de las funciones, velocidad del sistema,

confiabilidad de guardado) y satisfacción global (recomendación a otros docentes, disposición a sustituir plataformas actuales).

Observación directa: Un integrante del equipo fue asignado a cada grupo de 8–10 participantes con la instrucción de registrar verbalizaciones espontáneas, momentos de confusión visible y consultas realizadas al facilitador durante las tareas.

Hacer

Durante la prueba Alfa, el equipo ejecutó el plan de recopilación de información descrito en la fase anterior. A continuación, se presentan los principales resultados obtenidos:

Resultados del cuestionario post-sesión (participantes = 80):

- Claridad de la interfaz: promedio de 4.3/5. El 87% de los participantes calificó la organización de los elementos visuales como clara o muy clara.
- Facilidad de navegación: promedio de 4.1/5. El módulo de tareas fue el más elogiado; el módulo de evaluación (rúbricas) obtuvo la calificación más baja con un promedio de 3.6/5.
- Confiabilidad de guardado: promedio de 4.4/5. El 91% de los docentes manifestó sentir confianza en que sus acciones (calificaciones, retroalimentación) quedaban correctamente almacenadas, lo que representa una mejora sustancial respecto al 40% reportado en la encuesta diagnóstica inicial.
- Satisfacción global: el 78% de los participantes indicó que recomendaría MaxGrade a otros docentes; el 65% expresó disposición a sustituir la plataforma actual de su plantel por MaxGrade si estuviera disponible de forma institucional.

Tickets de soporte registrados durante las sesiones (23 en total):

- 9 tickets relacionados con la carga de archivos: usuarios que intentaron subir archivos superiores al límite de 50 MB o con formatos no compatibles. Resolución: se actualizó el mensaje de error para indicar explícitamente el límite y los formatos aceptados (PDF, DOCX, JPG, PNG).
- 6 tickets relacionados con la creación de rúbricas: docentes que no encontraban la opción de asociar una rúbrica existente a una nueva tarea. Resolución: se rediseñó el flujo de creación de tareas para mostrar la opción de rúbrica de manera más visible en el paso 2 del asistente.
- 5 tickets sobre notificaciones: alumnos que no recibían la notificación de tarea calificada. Resolución: se identificó un bug en el servicio de envío de notificaciones que omitía usuarios con ciertos caracteres especiales en el correo electrónico. Corregido y re-verificado.

- 3 tickets misceláneos: consultas sobre cómo restablecer contraseña, cómo archivar una clase y cómo usar el transcriptor de voz. Resolución: se incorporaron tooltips contextuales en las secciones correspondientes.

Hallazgos de los logs del sistema: El análisis de los registros de actividad reveló que el flujo de calificación de entregas presentaba una tasa de abandono del 12% en el paso de asociación de rúbrica, lo que confirmó el hallazgo de los tickets de soporte. Asimismo, se detectó que el tiempo promedio para completar la tarea de creación de clase fue de 2 minutos 14 segundos, por debajo del umbral de 3 minutos establecido como criterio de aceptación por el equipo.

Verificar

En esta fase, el equipo evaluó el grado de cumplimiento de los objetivos establecidos en la fase de Planificar, contrastando los resultados obtenidos con los criterios de aceptación previamente definidos.

Objetivo cumplido — Confiabilidad percibida: El indicador más crítico del proyecto era reducir la incertidumbre de los docentes respecto al guardado de calificaciones. El resultado de 4.4/5 en el cuestionario y el 91% de confianza reportada superan ampliamente el umbral mínimo de aceptación establecido en un 70%.

Objetivo parcialmente cumplido — Usabilidad del módulo de evaluación: Si bien la satisfacción general fue alta, el módulo de rúbricas obtuvo la calificación más baja del instrumento (3.6/5), por debajo del umbral de 4.0/5 definido como criterio de éxito. Esto indica que, aunque el módulo es funcional, su flujo de uso no es suficientemente intuitivo para el perfil de usuario objetivo.

Objetivo cumplido — Tiempo de completitud de tareas críticas: El tiempo promedio para la creación de clase (2:14 min), la entrega de tarea por parte del alumno (1:38 min) y la calificación con retroalimentación por parte del docente (3:42 min) se mantuvieron dentro de los umbrales de aceptación establecidos (máximo 5 minutos para cualquier tarea primaria).

Objetivo no alcanzado — Tasa de adopción espontánea del transcriptor de voz: Solo el 31% de los docentes utilizó la función de retroalimentación por voz durante las sesiones de prueba, por debajo del 50% esperado. Las observaciones directas sugieren que la mayoría de los docentes desconocía la existencia de esta función o no confiaba en la precisión del transcriptor. Este resultado se considera un área de mejora prioritaria.

Actuar

Con base en los hallazgos de la fase de Verificar, el equipo definió las siguientes acciones concretas para la siguiente iteración del sistema:

Rediseño del módulo de rúbricas: Se planea simplificar el flujo de creación y asociación de rúbricas, reduciendo el número de pasos de 4 a 2 mediante un asistente guiado. Adicionalmente, se incorporará una galería de rúbricas predefinidas por materia para que los docentes puedan utilizarlas directamente sin necesidad de crearlas desde cero.

Visibilidad del transcriptor de voz: Se rediseñará la ubicación del botón de dictado de retroalimentación para que sea visible de forma inmediata al abrir el panel de calificación de una entrega, acompañado de una etiqueta textual descriptiva en lugar del ícono actual. Se evaluará también la incorporación de un tutorial contextual de primer uso.

Optimización del sistema de notificaciones: Se realizará una revisión exhaustiva del servicio de envío de notificaciones para garantizar la compatibilidad con todos los formatos de correo electrónico, incluyendo aquellos con caracteres especiales o dominios institucionales del IPN.

Prueba Beta controlada: Como siguiente paso en el ciclo de validación, el equipo planea lanzar una versión Beta de MaxGrade con acceso abierto para todos los docentes del CECyT 9 durante el inicio del siguiente ciclo escolar. Esta prueba permitirá evaluar el comportamiento del sistema en condiciones de uso real, sin supervisión del equipo de desarrollo, y recopilará datos de uso durante un periodo de 4 semanas para alimentar la siguiente iteración del Ciclo de Deming.

Expansión institucional: A largo plazo, se contempla la posibilidad de presentar MaxGrade como solución oficial ante la Dirección General del IPN, con el objetivo de su adopción en otros planteles del sistema CECyT. Para ello, se deberá desarrollar un módulo de administración institucional que permita la gestión centralizada de planteles, docentes y periodos escolares.

Conclusiones

El desarrollo de MaxGrade representa una respuesta concreta a una problemática documentada y persistente en el entorno educativo del CECyT 9: la exclusión digital de los docentes derivada de plataformas de gestión académica que no han sido diseñadas con sus necesidades y capacidades en mente. A lo largo del proceso descrito en esta tesina, el equipo Uxmal demostró que es posible construir una herramienta tecnológica que combine funcionalidad robusta con simplicidad de uso, siempre que el diseño parta de una comprensión profunda del usuario final.

La metodología SCRUM adoptada para el desarrollo resultó adecuada para un proyecto de esta naturaleza, ya que permitió al equipo adaptarse a los cambios en los requerimientos, entregar incrementos funcionales de forma continua y mantener una comunicación activa a pesar de los retos de coordinación identificados en las retrospectivas. Los tres sprints ejecutados lograron cubrir la totalidad de las 49 historias de usuario planificadas, con una gestión efectiva de los 8 defectos identificados durante las pruebas funcionales.

La prueba de usuario Alfa realizada con 80 participantes del CECyT 9 proporcionó evidencia sólida sobre el impacto positivo de MaxGrade en la confianza tecnológica de los docentes. El incremento en la percepción de confiabilidad de guardado —del 40% reportado en la encuesta diagnóstica al 91% registrado en el cuestionario post-sesión— constituye el indicador más significativo del valor que la plataforma aporta a su público objetivo. De igual forma, la disposición del 78% de los participantes a recomendar la herramienta refuerza la pertinencia de la solución desarrollada.

La aplicación del Ciclo de Deming como marco de análisis de los resultados permitió al equipo ir más allá de la simple medición de satisfacción, identificando con precisión los módulos y flujos que requieren mejora, las causas raíz de los problemas reportados y las acciones concretas necesarias para la siguiente iteración. El módulo de rúbricas y la función de retroalimentación por voz emergieron como las áreas de desarrollo prioritario para la versión Beta del sistema.

Desde la perspectiva de los autores, uno de los aprendizajes más valiosos de este proyecto fue la confirmación de que el problema de adopción tecnológica en los docentes no es un problema de actitud ni de capacidad, sino fundamentalmente de diseño. Cuando una herramienta es realmente simple, clara y confiable, los usuarios —independientemente de su edad o experiencia tecnológica previa— son capaces de adoptarla con rapidez y de integrarla de manera natural a su práctica profesional.

MaxGrade no es un producto terminado: es el inicio de un ciclo de mejora continua. Los próximos pasos —la prueba Beta, el rediseño del módulo de rúbricas y la exploración de una adopción institucional a nivel IPN— representan la materialización del compromiso de Uxmal con su propia filosofía: construir tecnología hecha para durar, que sirva genuinamente a las personas y que evolucione con ellas.

Referencias.

Rodríguez López, J. (2017). *Diagnóstico sobre conocimientos y habilidades de los docentes sobre el uso y aplicación de las TIC en la Universidad* [Tesis de Licenciatura, Universidad Autónoma de Baja California]. Repositorio Institucional UABC.

Universidad Autónoma del Estado de Hidalgo. (2022). *Retos del docente ante el retorno a clases presenciales* [Tesis de Especialidad]. Repositorio de la Universidad Autónoma del Estado de Hidalgo.

Universidad Nacional Autónoma de México. (2019). *La alfabetización digital para adultos mayores* [Tesina de Licenciatura, Universidad Nacional Autónoma de México]. Repositorio de Tesis UNAM.

Yañez Castillo, J. A. (2017). *De lo efectivo a lo afectivo: análisis de la metáfora de escritorio e iconos para usuarios adultos mayores en México* [Tesis de Maestría, Universidad Autónoma Metropolitana]. Repositorio Institucional de la Universidad Autónoma Metropolitana.

Castañeda, S., y Ortega, I. (1999). Evaluación de estrategias de aprendizaje y orientación motivacional en estudiantes universitarios. *Revista de la Educación Superior*, 28(112). Publicaciones.anuies.mx

Comunicado de prensa: Las 10 excusas más comunes entre los adolescentes de hoy para no hacer la tarea | MyTutor. (n.d.). MyTutor for Schools. <https://schools.mytutor.co.uk/blog/press-release-todays-teens-top-10-excuses-missing-homework>

Delgado, P. (2025, December 15). *Cómo la curva del olvido afecta la enseñanza y la evaluación educativa*. Observatorio / Instituto Para El Futuro De La Educación. <https://observatorio.tec.mx/como-la-curva-del-olvido-afecta-la-ensenanza-y-la-evaluacion/>

Vista de La importancia del uso de la tecnología en el proceso educativo en México. (n.d.). <https://ciencialatina.org/index.php/cienciala/article/view/6780/10321>

Fuentes Documentales:

Universidad Madero. (s.f.). Tipos de maestros en México. Blog UMAD. Recuperado el [15 de abril de 2026], de <https://online.umad.edu.mx/blog/tipos-de-maestros-en-mexico>

Perochena González, P., Arteaga Martínez, B., Labatut Portilho, E. M., & Martínez Abad, F. (2017). Adaptación y validación del cuestionario: estilos de enseñanza (portilho/banas) en el contexto educativo español. *Tendencias pedagógicas*, (30), 71-90.

González, P. P., Martínez, B. A., Portilho, E. M. L., & Abad, F. M. (2017). Adaptación y validación del cuestionario: estilos de enseñanza (portilho/banas) en el contexto educativo español. Dialnet. <https://dialnet.unirioja.es/servlet/articulo?codigo=6164817>

Boluda, Ó. (13 de noviembre de 2015). Pirámide de necesidades del docente. efepeando. <https://www.efepeando.com/2015/11/piramide-de-necesidades-del-docente.html>

Araceli. (s. f.). Interacción humano-computadora: sus aplicaciones. <https://www.boletin.upiita.ipn.mx/index.php/ciencia/1013-cyt-numero-92/2085-interaccion-humano-computadora-sus-aplicaciones>

Glover, M. (2024, 26 febrero). What are UI elements? UI design elements explained | Uizard. The Uizard Blog | Learn More About AI-Powered UI Design! <https://uizard.io/blog/what-are-ui-elements-ui-design-elements-explained/>

Del Prado, J. A. (2023, 29 junio). Qué es un User Flow o flujo de usuario. UXABLES | Blog. <https://www.uxables.com/investigacion-ux/que-es-un-user-flow-o-flujo-de-usuario/>

ProductPlan. (2026, 21 abril). Intuitive Design | Glossary | ProductPlan. <http://productplan.com/glossary/intuitive-design>

Amazon Web Services. (s.f.). ¿Qué es un servidor? - Explicación de los servidores informáticos. AWS. <https://aws.amazon.com/es/what-is/server/>

Cisco Systems. (2022). ¿Qué es el intercambio de recursos entre dominios (CORS)? Cisco. https://www.cisco.com/c/es_mx/support/docs/security/adaptive-security-appliance-asas-215915-configure-and-troubleshoot-cors-on-asas.html

Google Safety Center. (s.f.). Cómo funciona la verificación en dos pasos con Google Authenticator. Google. <https://safety.google/authentication/>

IBM Cloud Education. (2021, 10 de mayo). Node.js: Una guía completa. IBM. <https://www.ibm.com/mx-es/topics/nodejs>

Microsoft Azure. (s.f.). Introducción a las arquitecturas de aplicaciones: Monolítica frente a microservicios. Microsoft. <https://learn.microsoft.com/es-es/azure/architecture/guide/architecture-styles/>

Mozilla Developer Network (MDN). (2023, 24 de julio). JavaScript: Guía de referencia del lenguaje. MDN Web Docs. <https://developer.mozilla.org/es/docs/Web/JavaScript>

Oracle Corporation. (s.f.). ¿Qué es una base de datos?. Oracle México. <https://www.oracle.com/mx/database/what-is-database/>

OpenJS Foundation. (s.f.). Documentación oficial de Express: El framework web minimalista para Node.js. Expressjs.com. <https://expressjs.com/es/>

Supabase Open Source. (2024). Documentación de Supabase: La alternativa de código abierto a Firebase. Supabase Docs. <https://supabase.com/docs>

Vercel Inc. (s.f.). Plataforma de despliegue y flujo de trabajo para desarrolladores. Vercel. <https://vercel.com/docs>

Anexos

- Anexo A: Encuesta-Profesores

https://correioipn.sharepoint.com/:w:/r/sites/6IV72026-1PI-LPTIIV-Equipo03/_layouts/15/Doc2.aspx?action=edit&sourcedoc=%7B3666d430-562b-4e7d-8ff8-fe2e12e68f91%7D&wdExp=TEAMS-TREATMENT&web=1